

Unit 2: Lexical Analysis

By Prashant Gautam

Outlines

- **Introduction**
- **Role of lexical analyzer**
- **Input buffering**
- **Specification and recognition of tokens**
- **Finite automata**
- **Conversion regular expression to NFA – Thomson's Construction**
- **NFA to DFA –subset construction**
- **Regular expression to DFA**
- **State minimization in DFA**
- **Flex/lex introduction.**

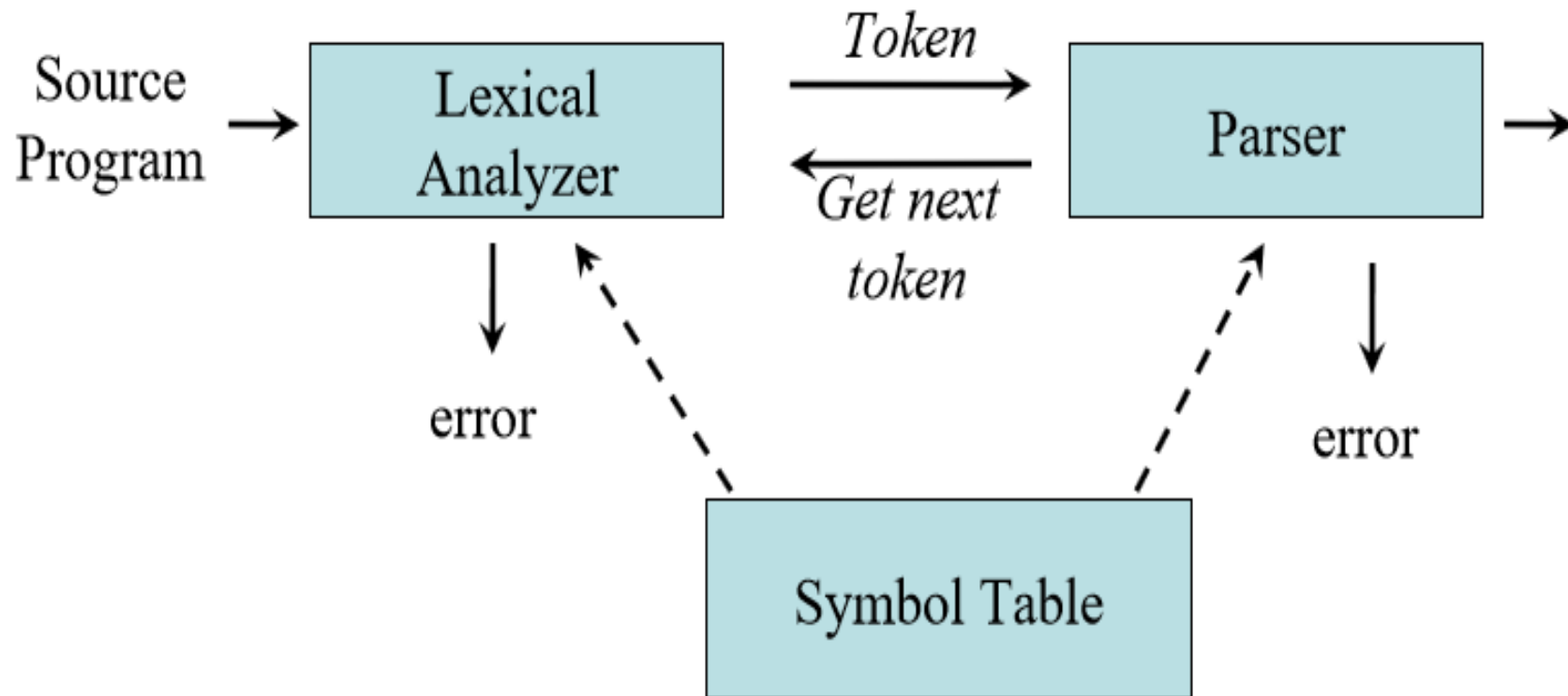
Lexical Analysis

- Lexical analysis is the process of converting a sequence of characters into a sequence of **tokens**.
- A program or function which performs lexical analysis is called a lexical analyzer or **scanner**.
- A **lexer** often exists as a single function which is called by a parser or another function.

Role of lexical analyzer

- The lexical analyzer is the first phase of a compiler.
- Its main task is to read the input characters and produce as output a sequence of tokens that the parser uses for syntax analysis.
- Lexical analyzer may also perform other auxiliary operation like removing redundant white space, removing token separator (like semicolon) etc.

- Upon receiving a “**get next token**” command from the parser, the lexical analyzer reads input characters until it can identify the next token.



Tokens, lexemes and Patterns

TOKENS

- A token is a string of characters, categorized according to the rules as a symbol (e.g., IDENTIFIER, NUMBER, COMMA).
- The process of forming tokens from an input stream of characters is called tokenization.
- A token can look like anything that is useful for processing an input text stream or text file.
- Consider this expression in the C programming language: **sum=3+2;**

Lexeme	Token type
sum	Identifier
=	Assignment operator
3	Number
+	Addition operator
2	Number
;	End of statement

LEXEME:

- Collection or group of characters forming tokens is called Lexeme.
- A token can represent more than one lexeme.

PATTERN:

- A pattern is a description of the form that the lexemes of a token may take.
- In the case of a keyword as a token, the pattern is just the sequence of characters that form the keyword.
- For example: “letter followed by letters and digits” and “non-empty sequence of digits”.
- Regular expressions are widely used to specify patterns.

Attributes of Tokens

- When a token represents more than one lexeme, lexical analyzer must provide additional information about the particular lexeme.
- This additional information is called as the attribute of the token.
- For simplicity, a token may have a single attribute which holds the required information for that token.
 - For identifiers, this attribute a pointer to the symbol table, and the symbol table holds the actual attributes for that token.
- Some attributes:
 - <id,attr> where attr is pointer to the symbol table
 - <assgop, _> no attribute is needed (if there is only one assignment operator)
 - <num,val> where val is the actual value of the number.

Example

dest= source + 5

- **Tokens:**
 - <id, pointer to symbol table entry for dest>,
 - <assignop>,
 - <id, pointer to symbol table entry for source>,
 - <num, 5>
- Token type and its attribute uniquely identifies a lexeme.

Error Recovery Strategies in LA

The following are the error-recovery actions in lexical analysis:

- 1) Deleting an extraneous(irrelevant) character.
- 2) Inserting a missing character.
- 3) Replacing an incorrect character by a correct character.
- 4) Transforming two adjacent characters.
- 5) **Panic mode recovery:** Deletion of successive characters from the token until error is resolved.

Look ahead and Input-Buffering

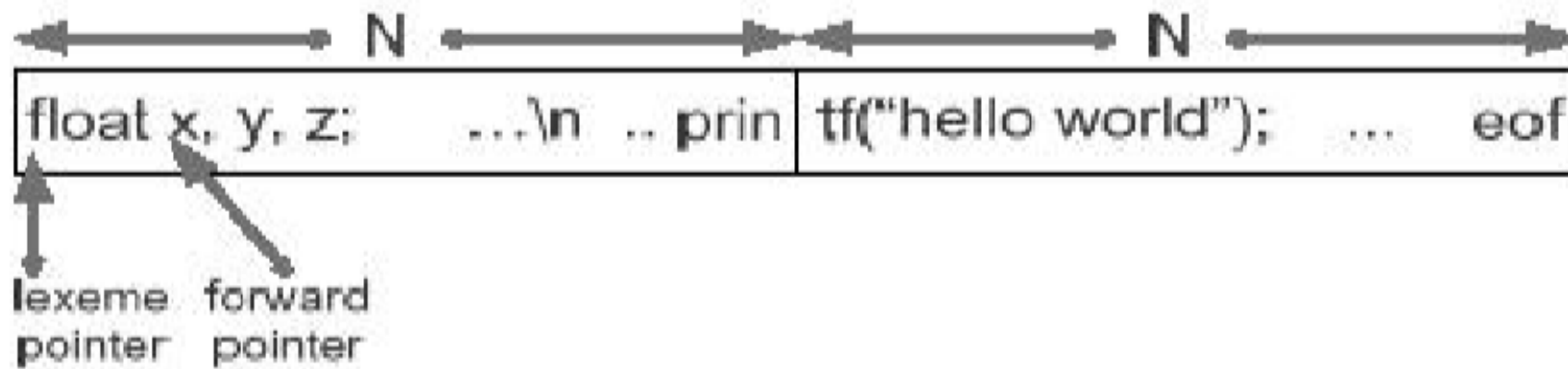
- Many times, a scanner has to look ahead several characters from the current character in order to recognize the token.

For example

int is keyword in C, while the term *inp* may be a variable name.

When the character 'i' is encountered, the scanner can not decide whether it is a keyword or a variable name until it reads two more characters.

In order to efficiently move back and forth in the input stream, input buffering is used



- Here, we divide the buffer into two halves with N -characters each.
- Rather than reading character by character from file we read N input character at once.
- If there are fewer than N characters in input **eof** marker is placed.
- There are two pointers (see in above fig.) the portion between lexeme pointer and forward pointer is current lexeme.
- Once the match for pattern is found, both the pointers points at the same place and forward pointer is moved

Forward Pointer

- The forward pointer performs tasks like below:

```
If forward at end of first half then,  
    Reload second half  
    Forward++  
end if  
else if forward at end of second half then,  
    Reload first half  
    Forward=start of first half  
end else if  
else  
    forward++
```

Specification of Tokens

There are 3 specifications of tokens:

- 1) Strings
- 2) Language
- 3) Regular expression

Strings and Languages

- An **alphabet** or character class is a finite set of symbols.
- A **string** over an alphabet is a finite sequence of symbols drawn from that alphabet.
- A **language** is any countable set of strings over some fixed alphabet.
- In language theory, the terms "sentence" and "word" are often used as synonyms for "string."
- The length of a string s , usually written $|s|$, is the number of occurrences of symbols in s .
- For example, banana is a string of length six.
- The empty string, denoted ϵ , is the string of length zero.

Operations on strings

1. A prefix of string s is any string obtained by removing zero or more symbols from the end of string s .

For example, **ban** is a prefix of **banana**.

2. A suffix of string s is any string obtained by removing zero or more symbols from the beginning of s .

For example, **nana** is a suffix of **banana**.

3. A substring of s is obtained by deleting any prefix and any suffix from s .

For example, **nan** is a substring of **banana**.

4. The **proper prefixes**, **suffixes**, and **substrings** of a string s are those prefixes, suffixes, and substrings, respectively of s that are not ϵ or not equal to s itself.

5. A subsequence of s is any string formed by deleting zero or more not necessarily consecutive positions of s .

For example, **baan** is a subsequence of **banana**.

Operations on languages

- The following are the operations that can be applied to languages:

1.Union

2.Concatenation

3.Kleene closure

4.Positive closure

Example : Operations on language

- Let $L=\{0, 1\}$ and $S=\{a, b, c\}$

1. Union : $L \cup S=\{0,1,a,b,c\}$

2. Concatenation: $L.S=\{0a,1a,0b,1b,0c,1c\}$

3. Kleene closure : $L^*=\{ \epsilon, 0, 1, 00, \dots \}$

4. Positive closure : $L^+=\{0,1,00, \dots \}$

Regular Expressions

- Each regular expression r denotes a language $L(r)$.
- Here are the rules that define the regular expressions over some alphabet Σ and the languages that those expressions denote:
 1. ϵ is a regular expression, and $L(\epsilon)$ is $\{ \epsilon \}$, that is, the language whose sole member is the empty string.
 2. If ' a ' is a symbol in Σ , then ' a ' is a regular expression, and $L(a) = \{a\}$, that is, the language with one string, of length one, with ' a ' in its one position.

3. Suppose r and s are regular expressions denoting the languages $L(r)$ and $L(s)$. Then,

a) $(r)|(s)$ is a regular expression denoting the language $L(r) \cup L(s)$.

b) $(r)(s)$ is a regular expression denoting the language $L(r)L(s)$.

c) $(r)^*$ is a regular expression denoting $(L(r))^*$.

d) (r) is a regular expression denoting $L(r)$.

Precedence

- 4. The unary operator * has highest precedence
- 5. Concatenation has second highest precedence and
- 6. | has lowest precedence and is left associative.

Practice Session

- Given the alphabet $A = \{0, 1\}$

1. RE denotes the language of all string that begins with a '1' and ends with a '0'. →

$1(1+0)^*0$

2. RE denotes the language of all strings that ends with 00 (binary number multiple of 4) →

$(1+0)^*00$

3. RE denotes the set of all stings of even length that describe alternating 1s and 0s →

$(01)^* + (10)^*$

4. RE denotes the string having exactly three 1's. →

$(0^* 1 0^* 1 0^* 1 0^*)$

5. RE denotes the string having at most two 0's →

$1^*(0 + \epsilon)1^*(0 + \epsilon)1^*$

6. RE denotes the regular expression to specify the identifier like in C →

$(A | B | C | | Z | a | b | c | | z | _) . ((A | B | C | | Z | a | b | c | | z | _) (0 | 1 | 2 | | 9))^*$

7. RE denotes string having substring 001 →

$(1+0)^* 001 (1+0)^*$

Regular Definitions

- Giving names to regular expressions is referred to as a Regular definition. If Σ is an alphabet of basic symbols, then a regular definition is a sequence of definitions of the form

$d_1 \rightarrow r_1$

$d_2 \rightarrow r_2$

.....

$d_n \rightarrow r_n$

Where,

1. Each d_i is a distinct name.
2. Each r_i is a regular expression over the alphabet $\Sigma \cup \{d_1, d_2, \dots, d_i\}$.

Example

- Regular definition for specifying identifiers in a programming language like C

letter $\rightarrow A \mid B \mid C \mid \dots \mid Z \mid a \mid b \mid c \mid \dots \mid z$

underscore $\rightarrow \text{'_}'$

digit $\rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$

id $\rightarrow (\text{letter} \mid \text{underscore}).(\text{letter} \mid \text{underscore} \mid \text{digit})^*$

Note: If we are trying to write the regular expression representing identifiers without using regular definition, it will be complex. $(A \mid B \mid C \mid \dots \mid Z \mid a \mid b \mid c \mid \dots \mid z \mid _) \cdot ((A \mid B \mid C \mid \dots \mid Z \mid a \mid b \mid c \mid \dots \mid z \mid _) (1 \mid 2 \mid \dots \mid 9))^*$

Practice Session

- Write regular definition for specifying floating point number in a programming language like C.

Soln:

digit $\rightarrow$ 0 | 1 | 2 | | 9

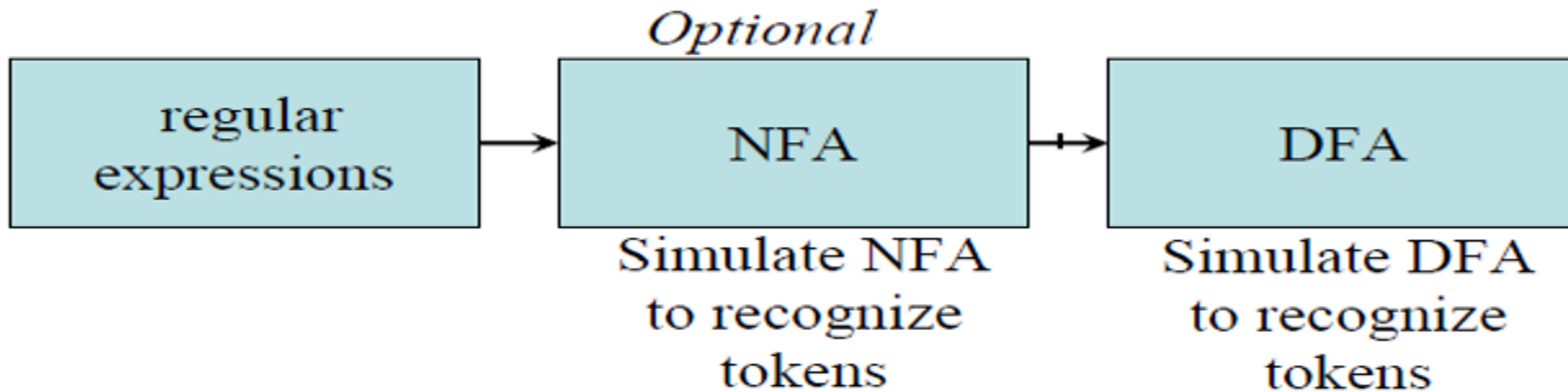
fnum $\rightarrow$ digit * (.digit+)

Design of a Lexical Analyzer

- First, we define regular expressions for tokens; then we convert them into a DFA to get a lexical analyzer for our tokens.

Algorithm1: Regular Expression $\rightarrow$ NFA $\rightarrow$ DFA (two steps: first to NFA, then to DFA)

Algorithm2: Regular Expression $\rightarrow$ DFA (directly convert a regular expression into a DFA)



Non-Deterministic Finite Automaton (NFA)

- FA is non deterministic, if there is more than one transition for each (state, input) pair.
- It is slower recognizer but it make take less spaces.
- An NFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$

where

Q is a finite set of states

Σ is a finite set of symbols

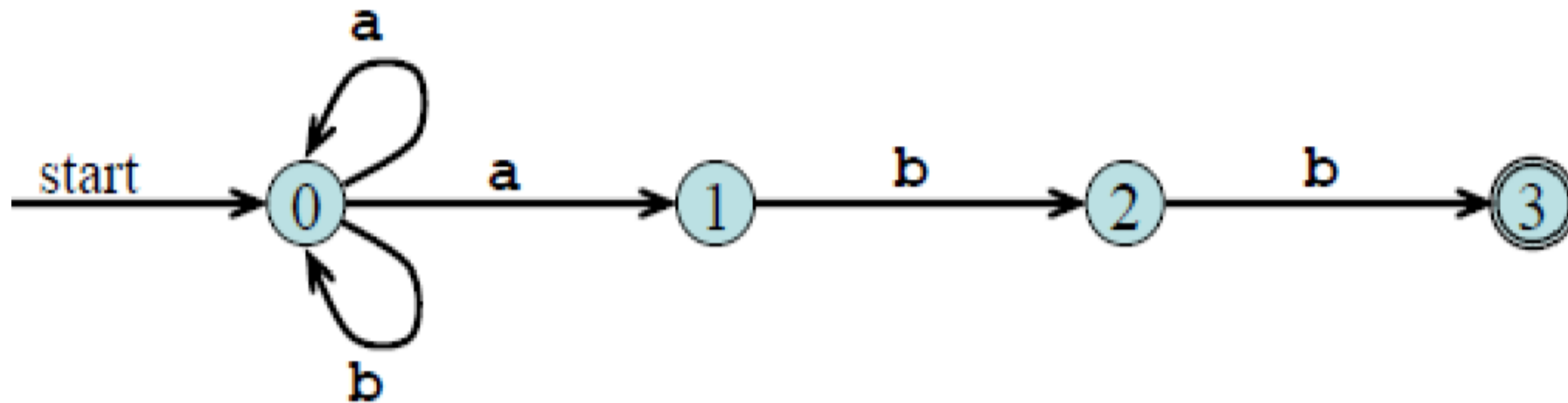
δ is a transition function where $\delta: Q \times \Sigma \rightarrow 2^Q$

$q_0 \in Q$ is the start state

$F \subseteq S$ is the set of accepting (or final) states

Example

A NFA accepts a string x , if and only if there is a path from the starting state to one of accepting states.



$$S = \{0,1,2,3\}$$

$$\Sigma = \{a,b\}$$

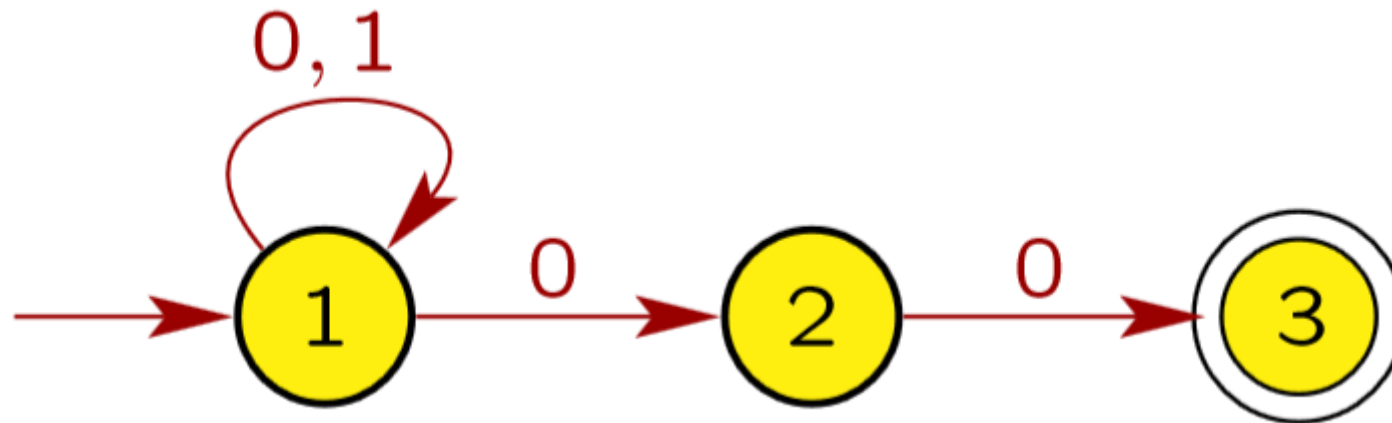
$$s_0 = 0$$

$$F = \{3\}$$

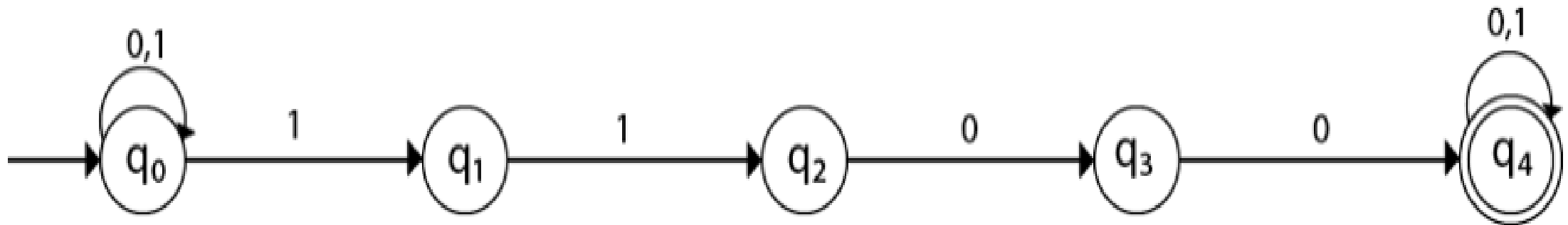
Fig: - NFA for regular expression $(a + b)^*a b b$

Practice

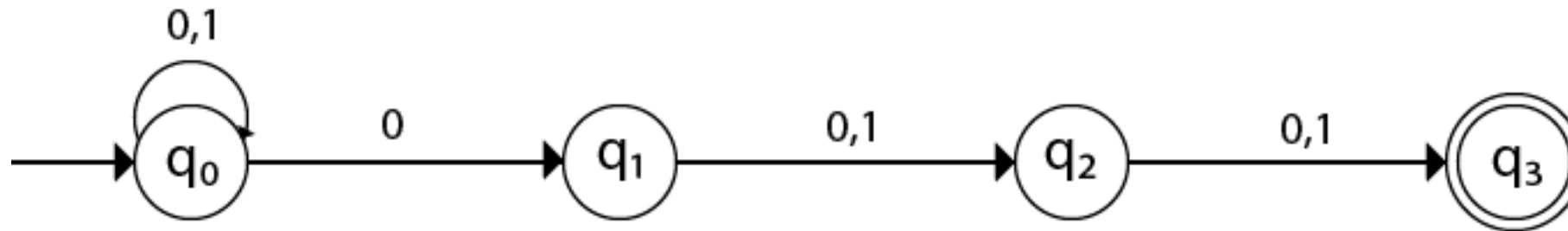
- NFA that accepts the language $\{w \in \Sigma^* \mid w \text{ ends with } 00\}$ with three states (the alphabet is $\Sigma = \{0,1\}$)



- Design an NFA with $\Sigma = \{0, 1\}$ in which double '1' is followed by double '0'.



- Design an NFA with $\Sigma = \{0, 1\}$ accepts all string in which the third symbol from the right end is always 0.



ϵ - NFA

- In NFA if a transition made without any input symbol is called ϵ -NFA. Here we need ϵ -NFA because the regular expressions are easily convertible to ϵ -NFA.

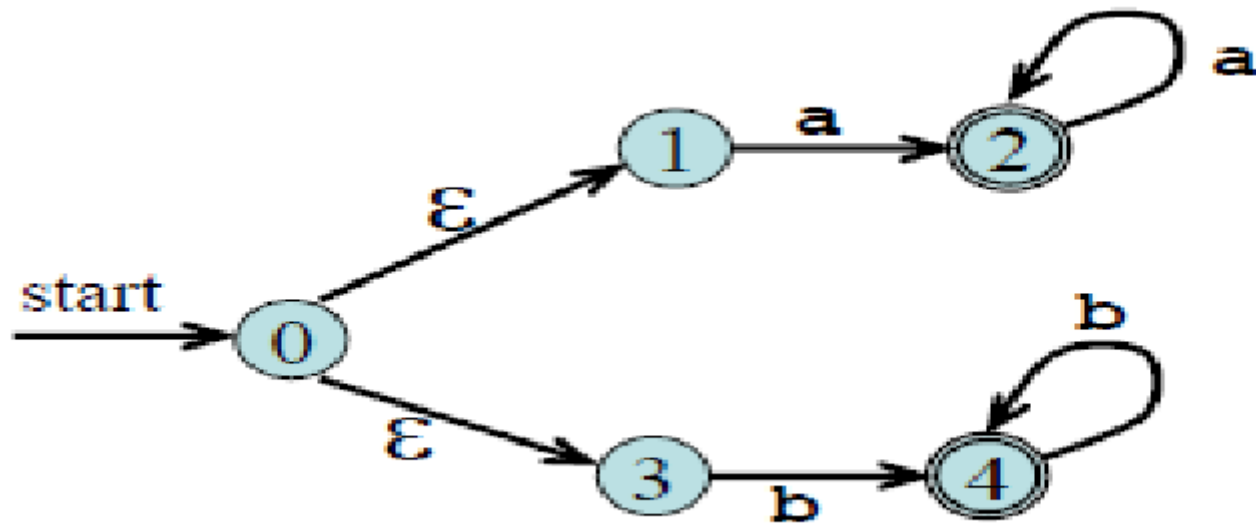


Fig: - ϵ -NFA for regular expression $aa^* + bb^*$

Deterministic Finite Automaton (DFA)

- DFA is a special case of NFA.
- There is only difference between NFA and DFA is in the transition function.
 - In NFA transition from one state to multiple states take place
 - In DFA transition from one state to only one possible next state take place.

Formal Definition

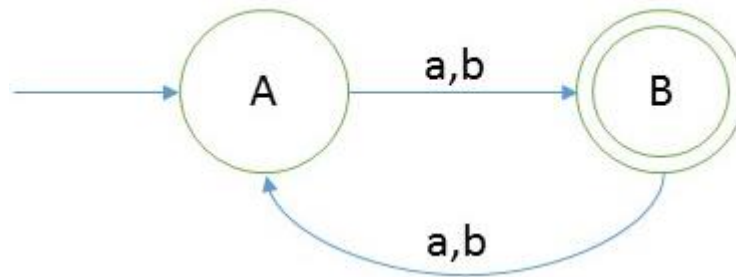
A DFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$

where –

- Q is a finite set of states.
- Σ is a finite set of symbols called the alphabet.
- δ is the transition function where $\delta: Q \times \Sigma \rightarrow Q$
- q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- F is a set of final state/states of Q ($F \subseteq Q$).

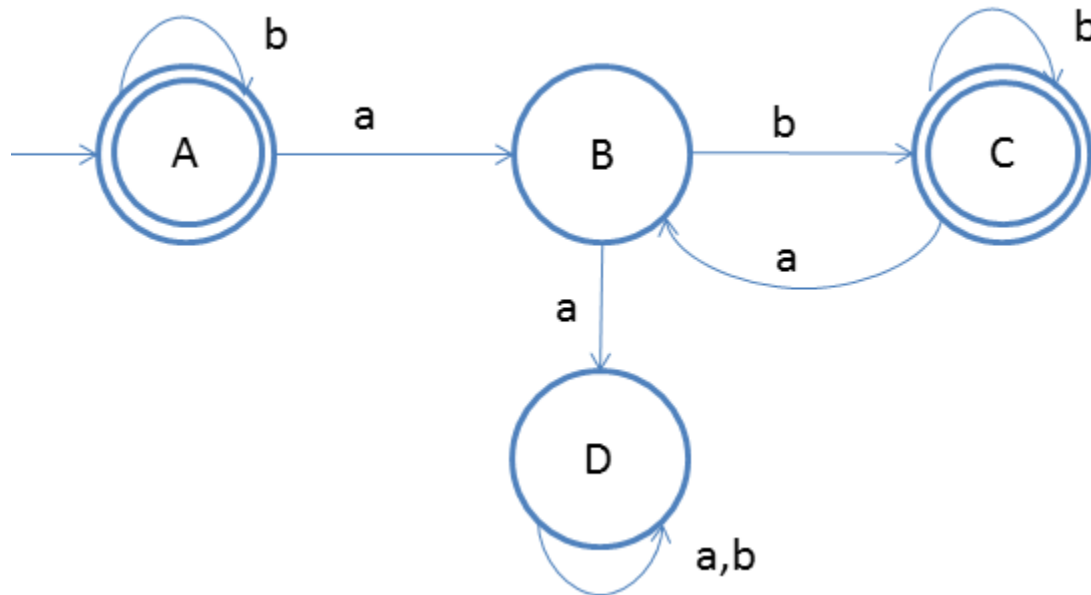
Examples

Create a DFA which accepts strings of odd length ($\Sigma = \{a, b\}$)

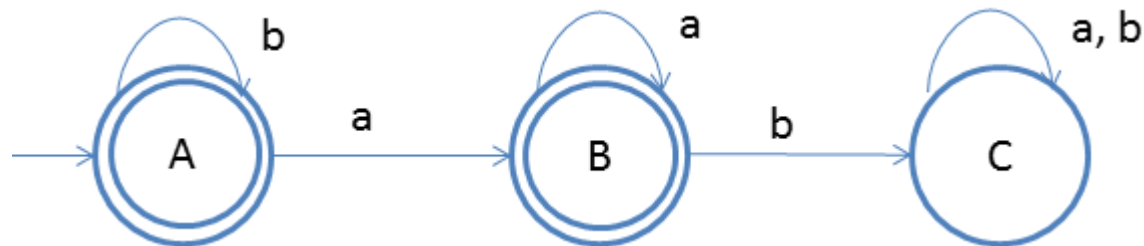


Practice Session

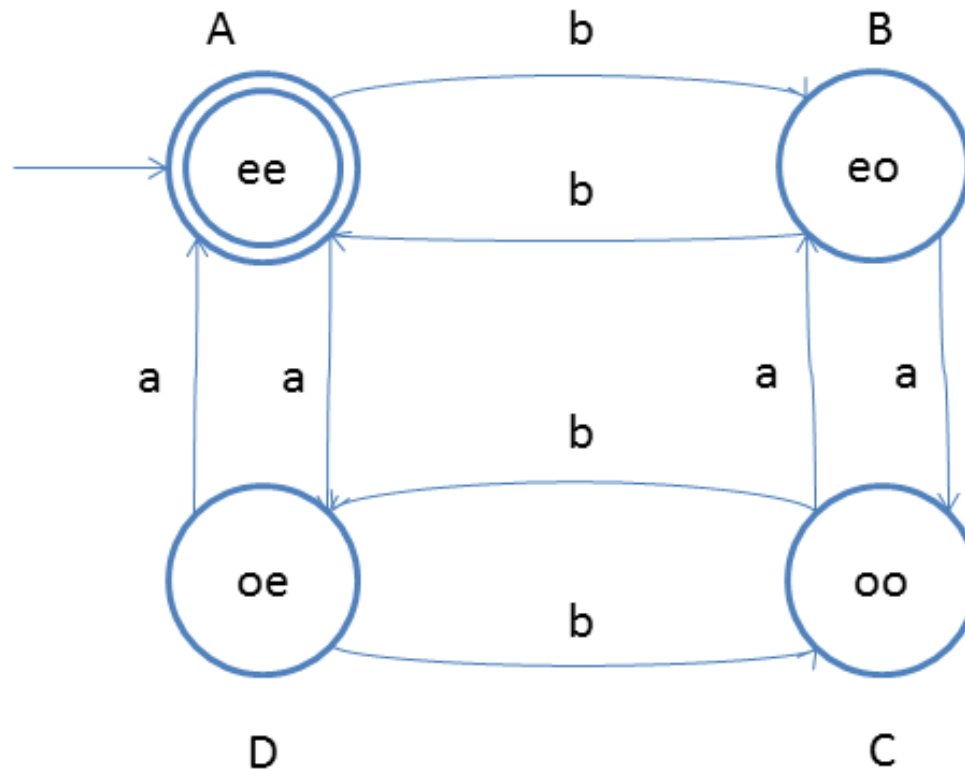
- Design a DFA in which every 'a' should be followed by 'b'
Given: Input alphabet, $\Sigma = \{a, b\}$
Language $L = \{\epsilon, ab, abab, bbbb, \dots\}$



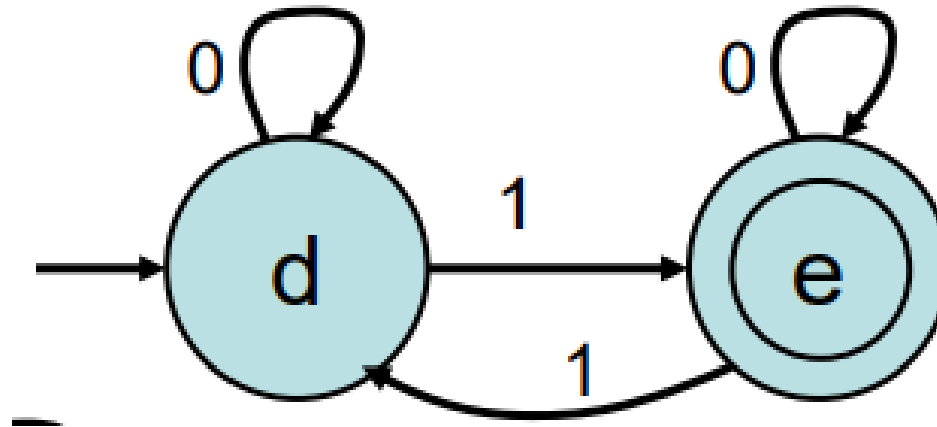
- Design a DFA in which every 'a' should never followed by 'b'
Given: Input alphabet, $\Sigma = \{a, b\}$
Language $L = \{\epsilon, a, aa, aaa, ba, \dots\}$



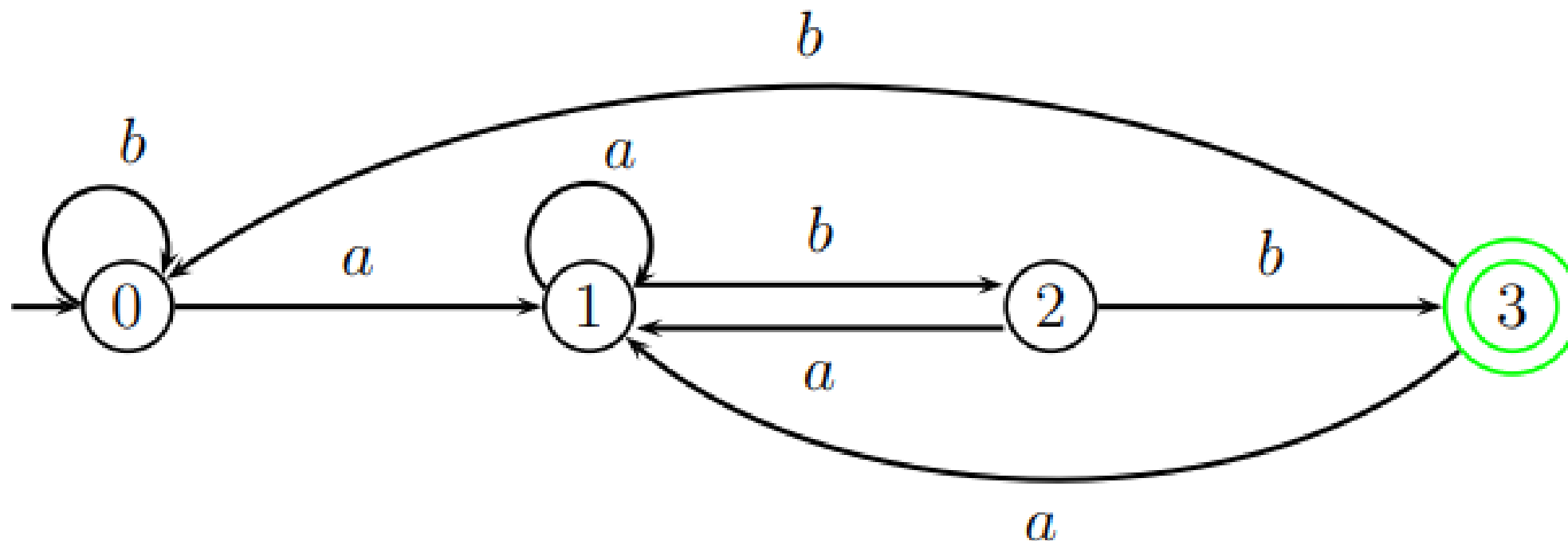
- Design a DFA Which accepts even numbers of 'a' and even numbers of 'b'



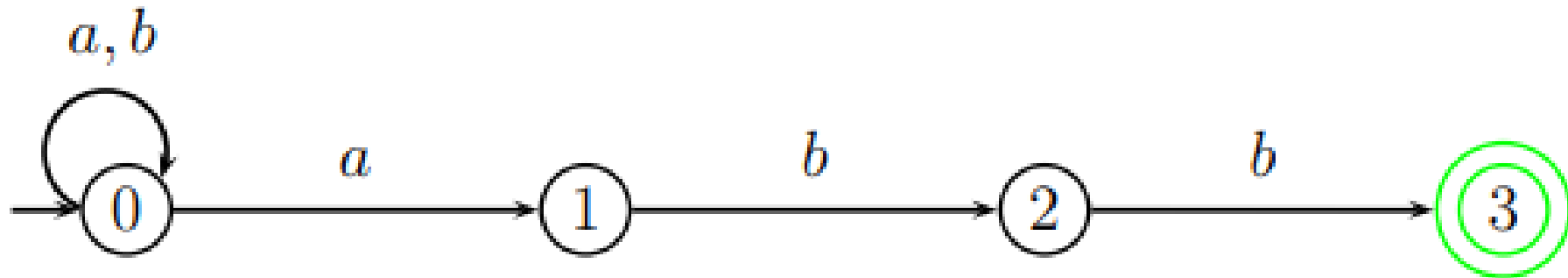
DFA: Odd number of 1's



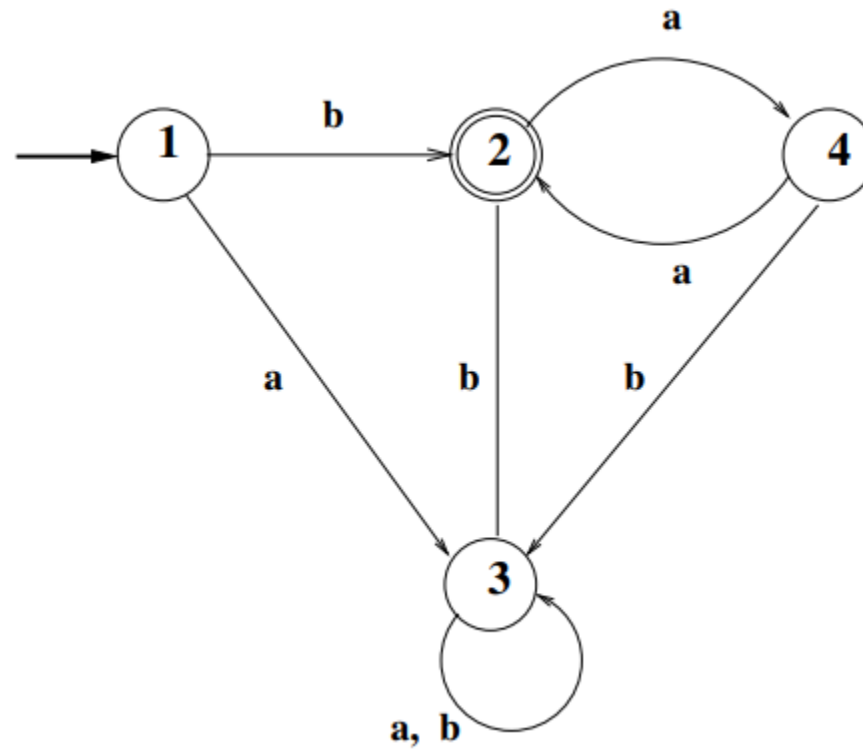
DFA for $\{a, b\}^*\{abb\}$



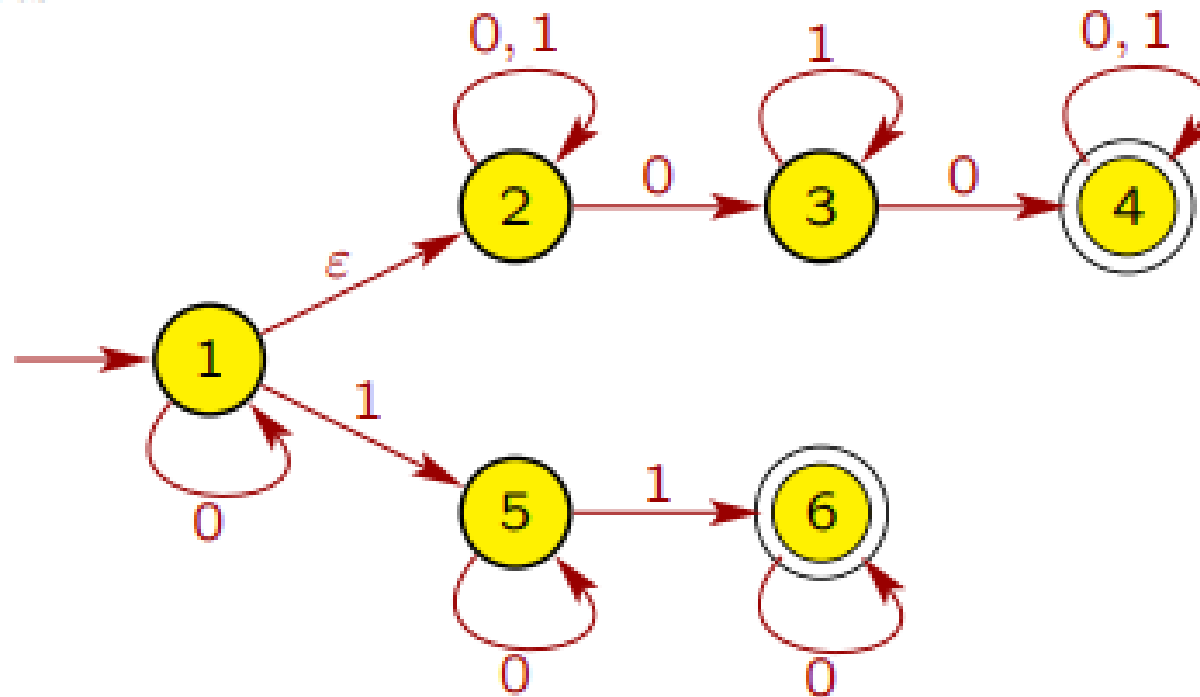
NFA for $\{a, b\}^*\{abb\}$



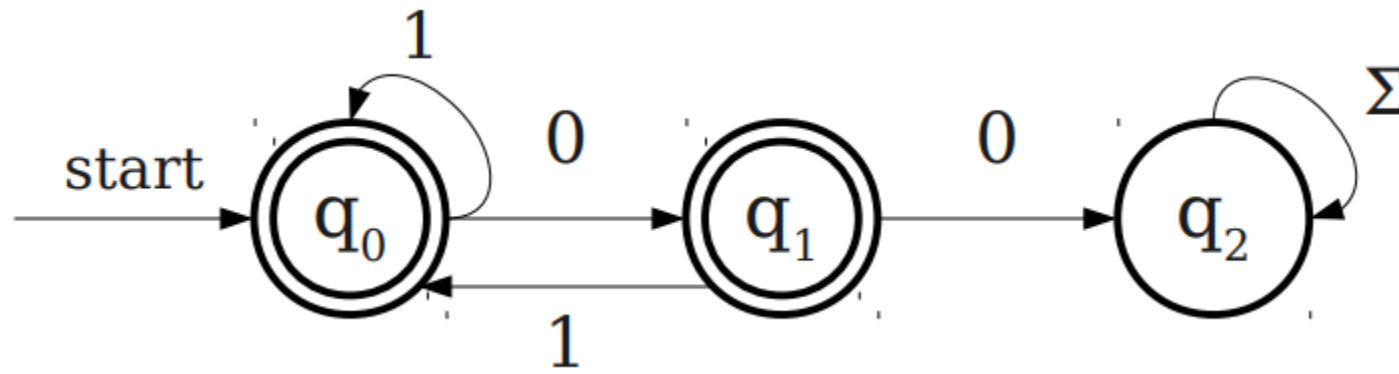
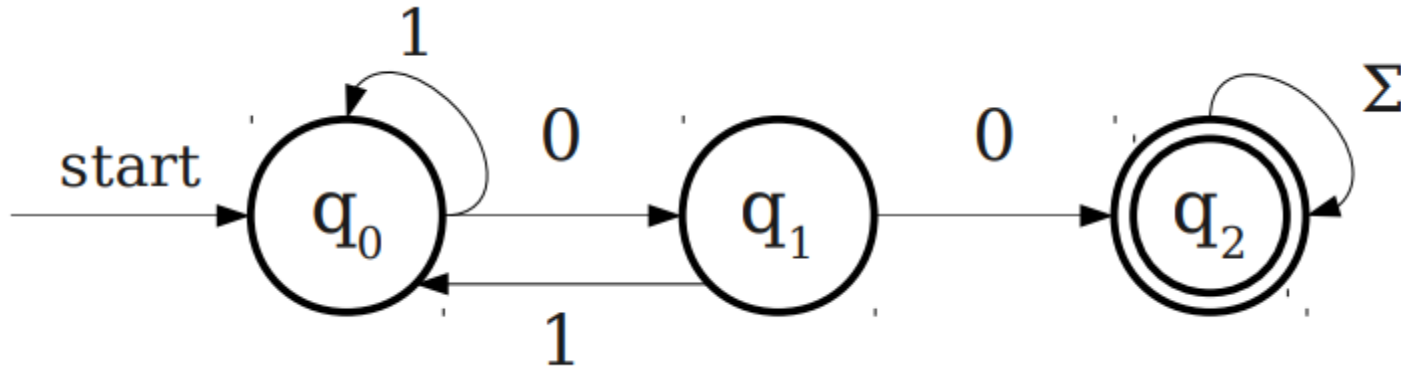
DFA : $b(aa)^*$



NFA: The language $\{ w \in \Sigma^* \mid w \text{ contains at least two 0s, or exactly two 1s} \}$ with six states.



$L = \{ w \in \{0, 1\}^* \mid w \text{ does not contain } 00 \text{ as a substring} \}$



$L = \{ w \mid w \text{ is a C-style comment} \}$

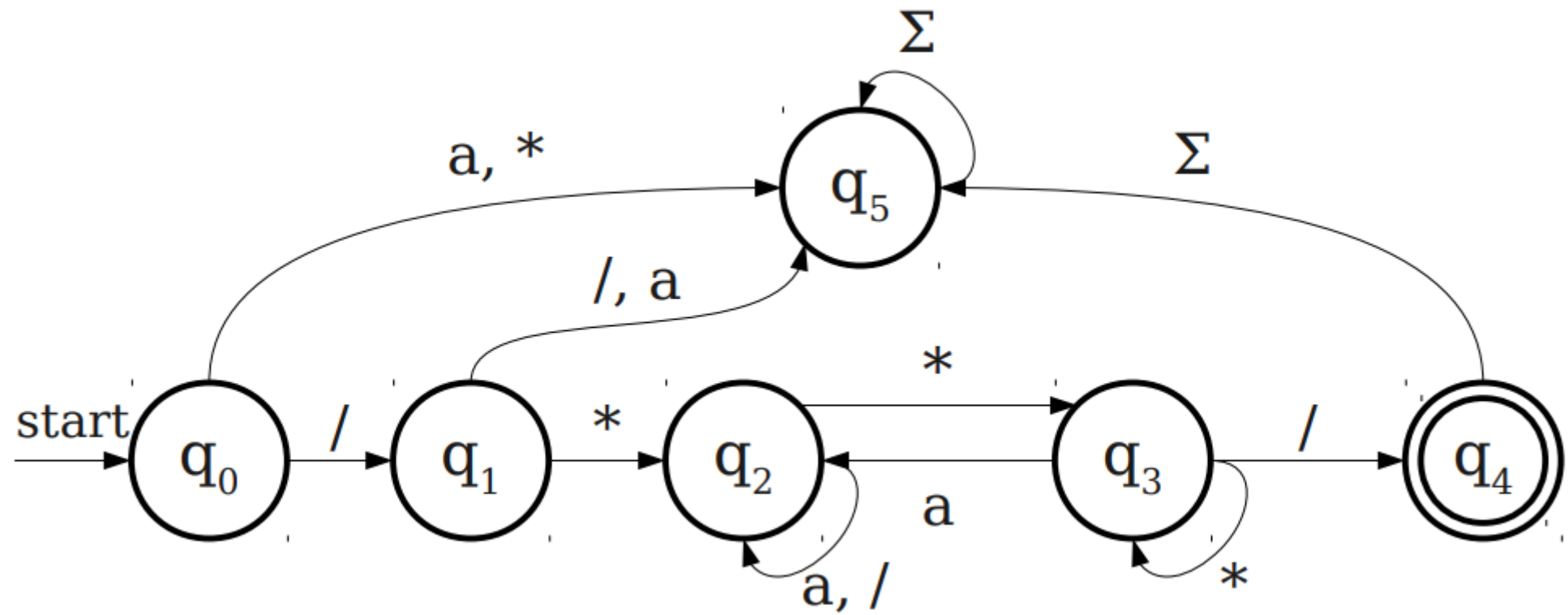
- Suppose $\Sigma = \{ a, *, / \}$

ACCEPTED

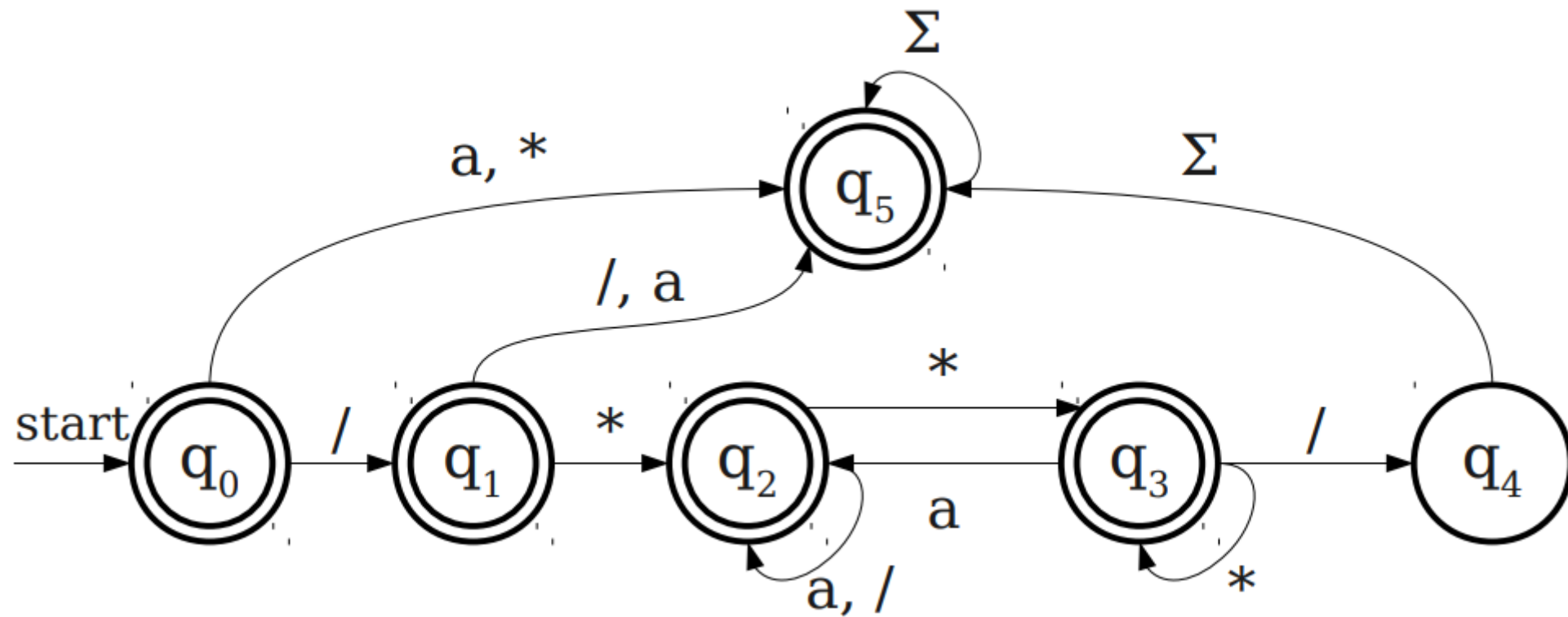
`/*a*/`
`/**/`
`/***/`
`/*aaa*aaa*/`

REJECTED

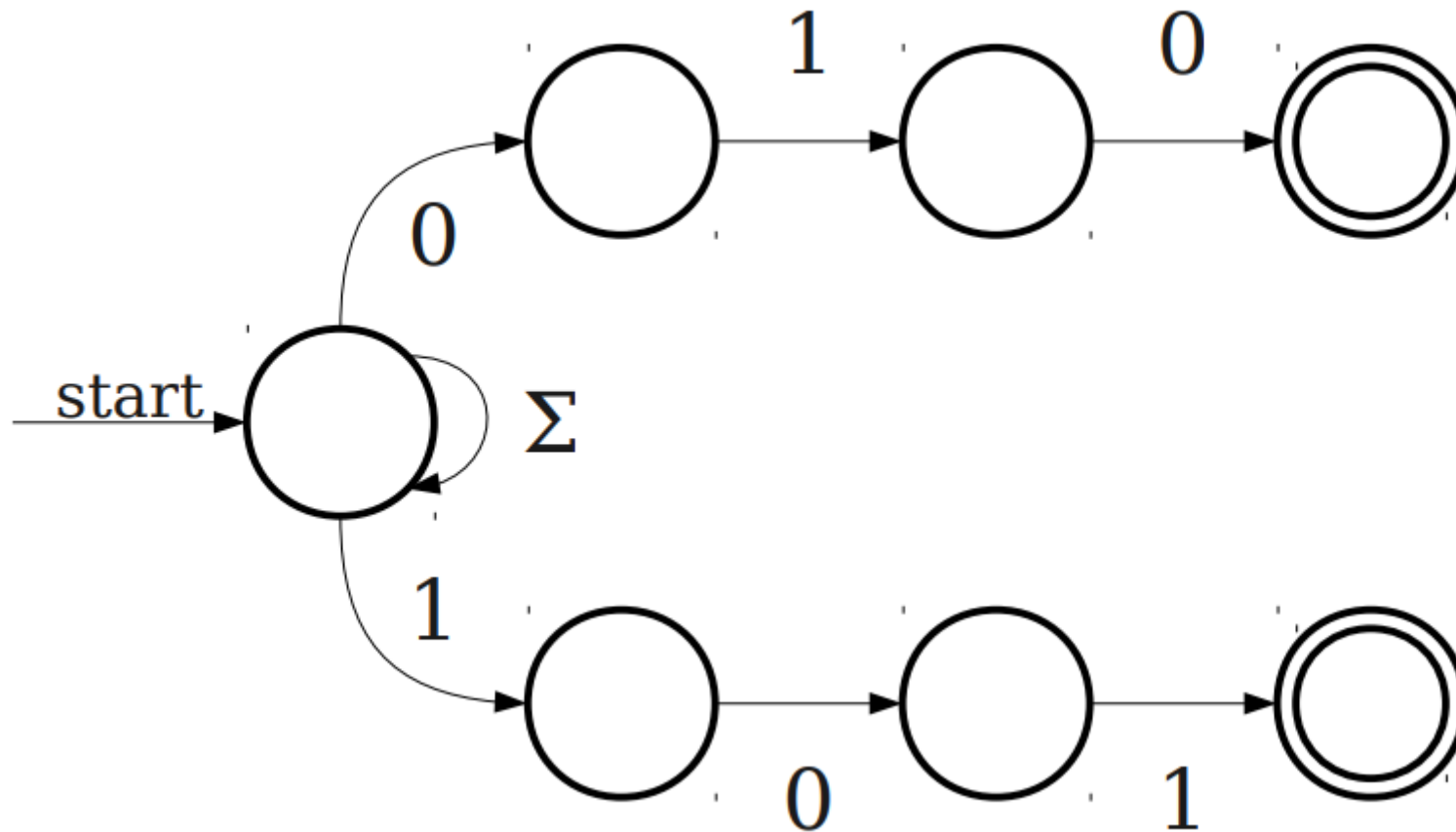
`/**`
`/**/a`
`aaa/**/`
`/*/`



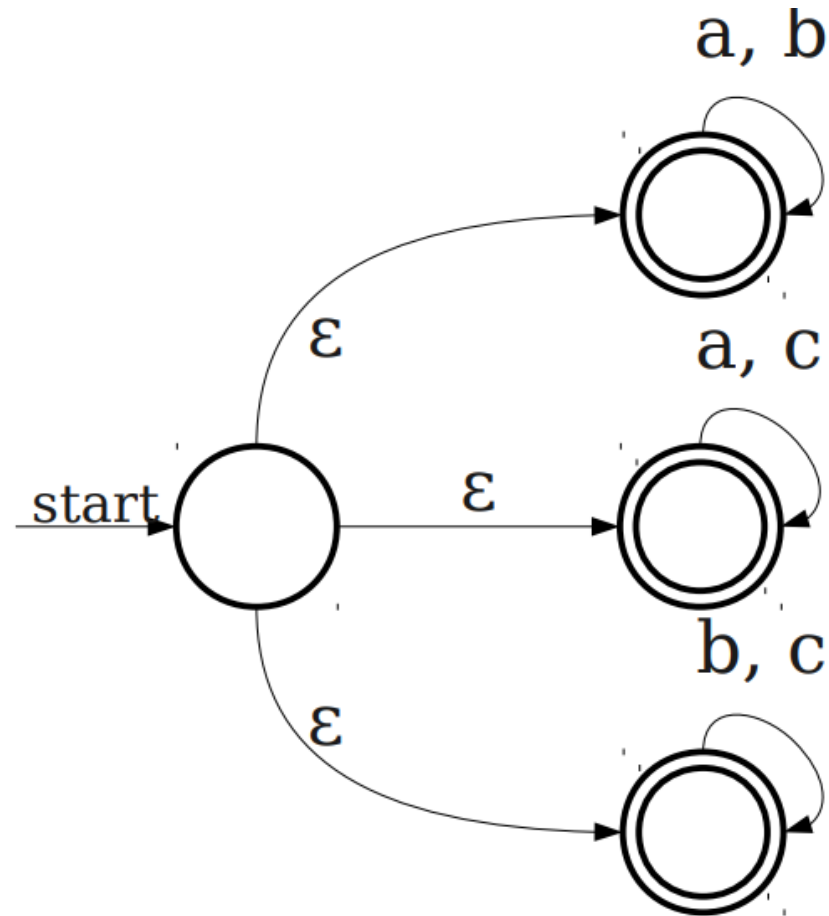
$L = \{ w \mid w \text{ is not a C-style comment} \}$



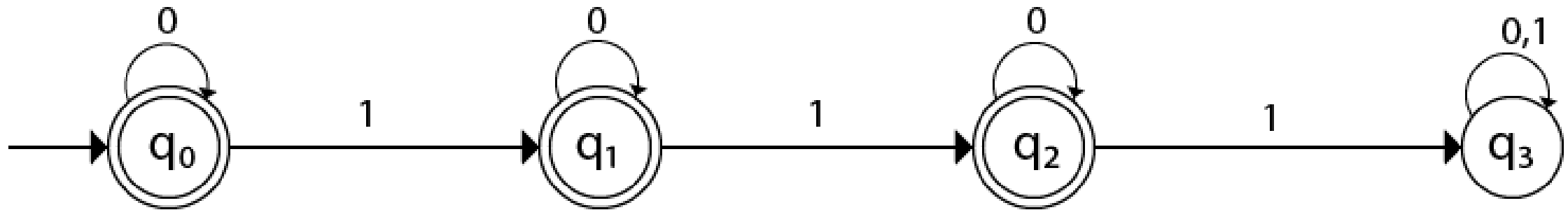
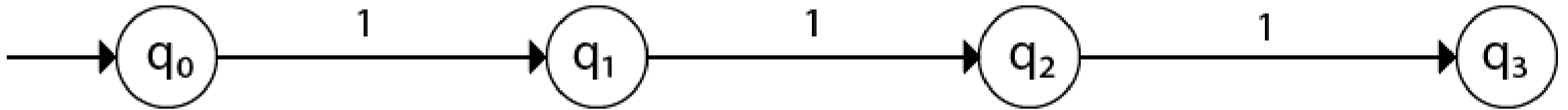
NFA: $L = \{ w \in \{0, 1\}^* \mid w \text{ ends in } 010 \text{ or } 101 \}$



NFA: $L = \{ w \in \{a, b, c\}^* \mid \text{at least one of } a, b, \text{ or } c \text{ is not in } w \}$



Design a DFA $L(M) = \{w \mid w \in \{0, 1\}^*\}$ and W is a string that does not contain consecutive 1's.



Conversion: Regular Expression to NFA

- Thomson's Construction

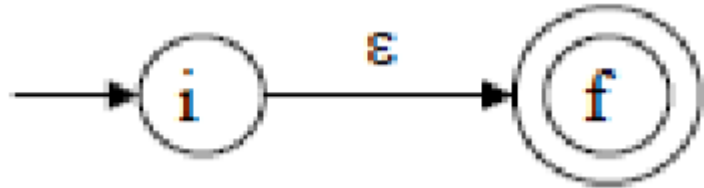
Thomson's Construction is simple and systematic method. It guarantees that the resulting NFA will have exactly one final state, and one start state.

Method:

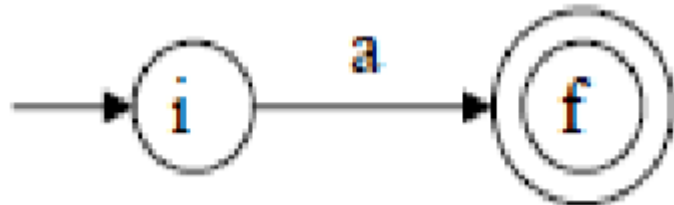
- First parse the regular expression into sub-expressions
- Construct NFA's for each of the basic symbols in regular expression
- Finally combine all NFA's of sub-expressions and we get required NFA of given regular expression.

Rules

1. To recognize an empty string ϵ

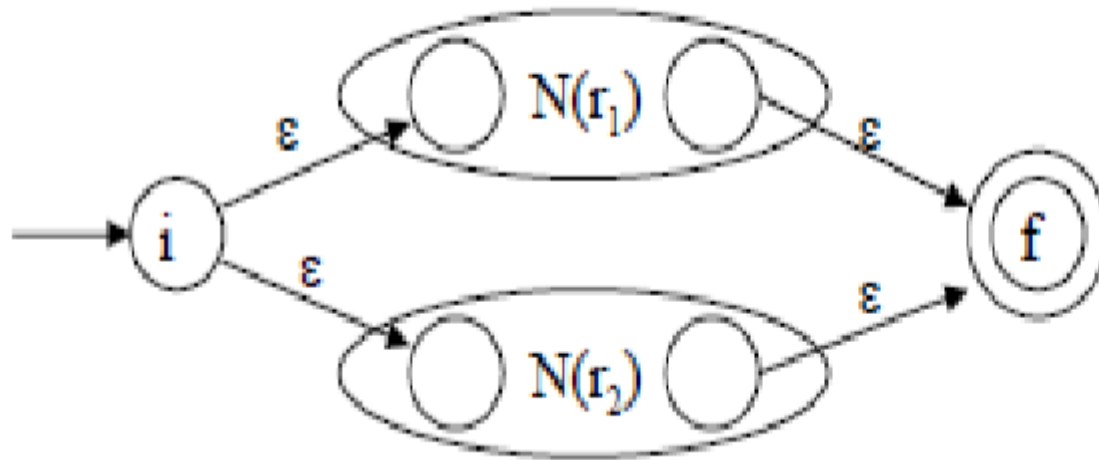


2. To recognize a symbol a in the alphabet Σ



3. If $N(r_1)$ and $N(r_2)$ are NFAs for regular expressions r_1 and r_2

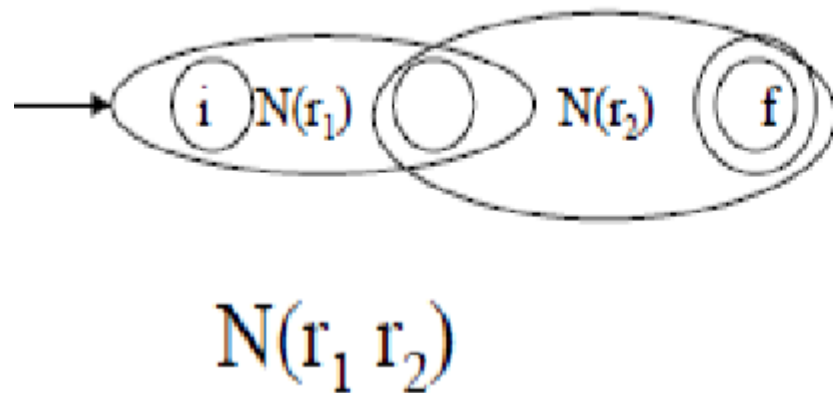
a) For regular expression $r_1 + r_2$



$N(r_1 + r_2)$

3.

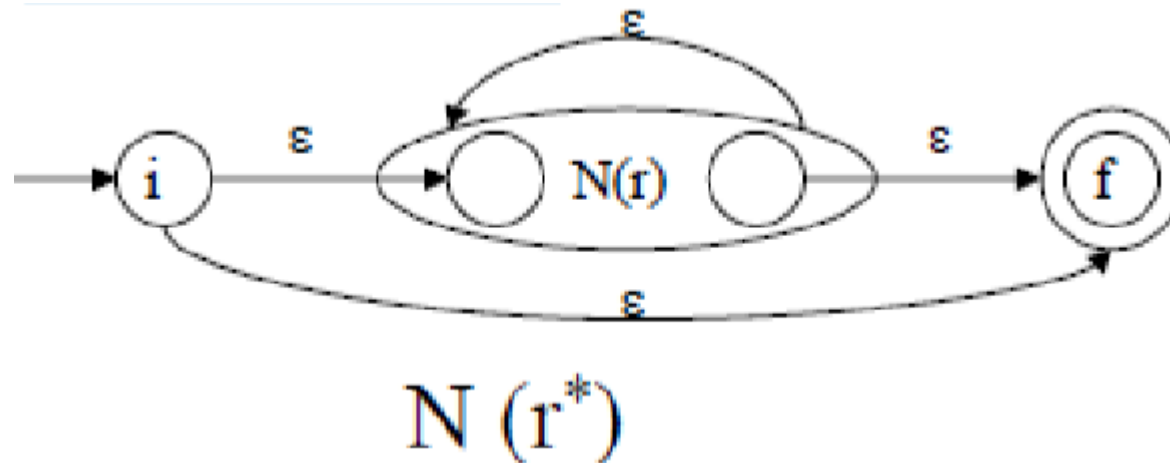
b) For regular expression $r_1 r_2$



The start state of $N(r_1)$ becomes the start state of $N(r_1 r_2)$ and final state of $N(r_2)$ become final state of $N(r_1 r_2)$

3.

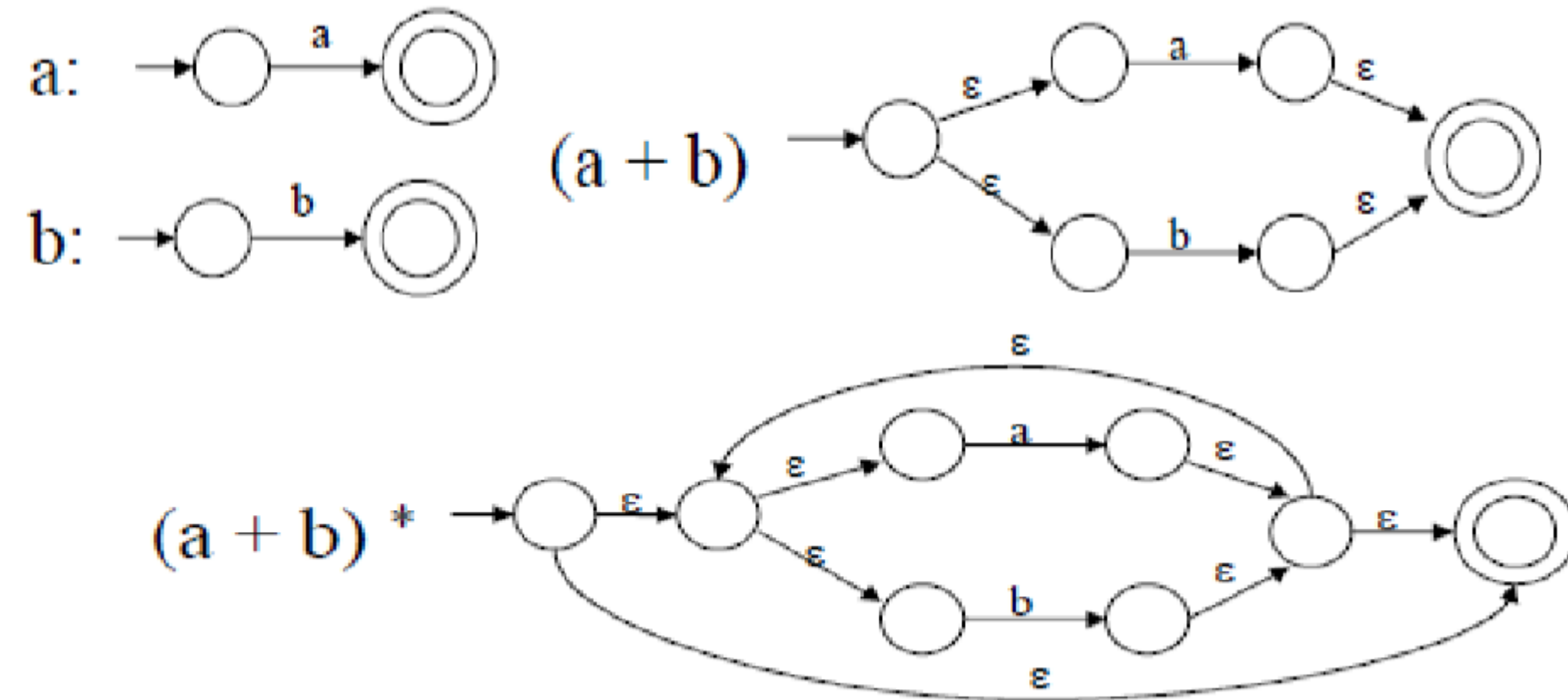
c) For regular expression r^*

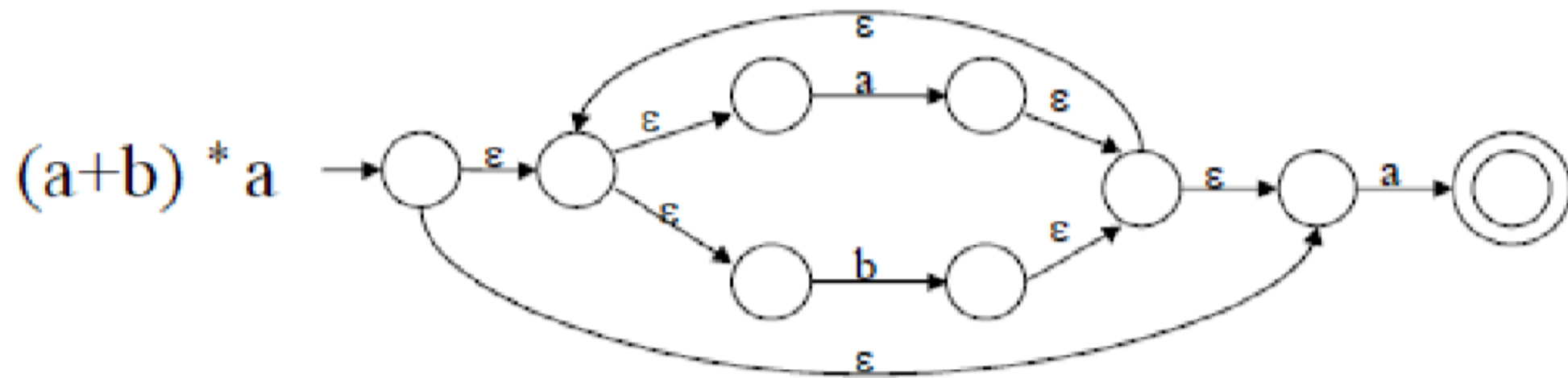


Using rule 1 and 2 we construct NFA's for each basic symbol in the expression, we combine these basic NFA using rule 3 to obtain the NFA for entire expression.

Example

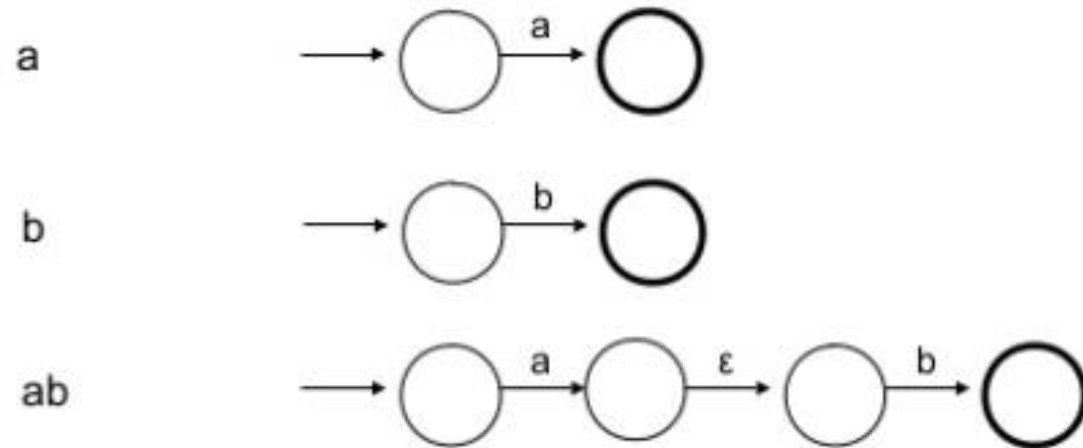
- NFA construction of RE $(a + b)^* a$



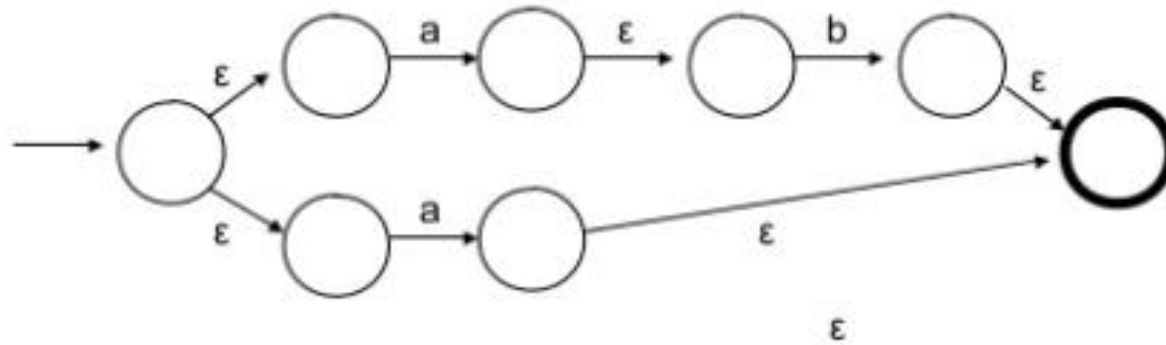


TRY Yourself

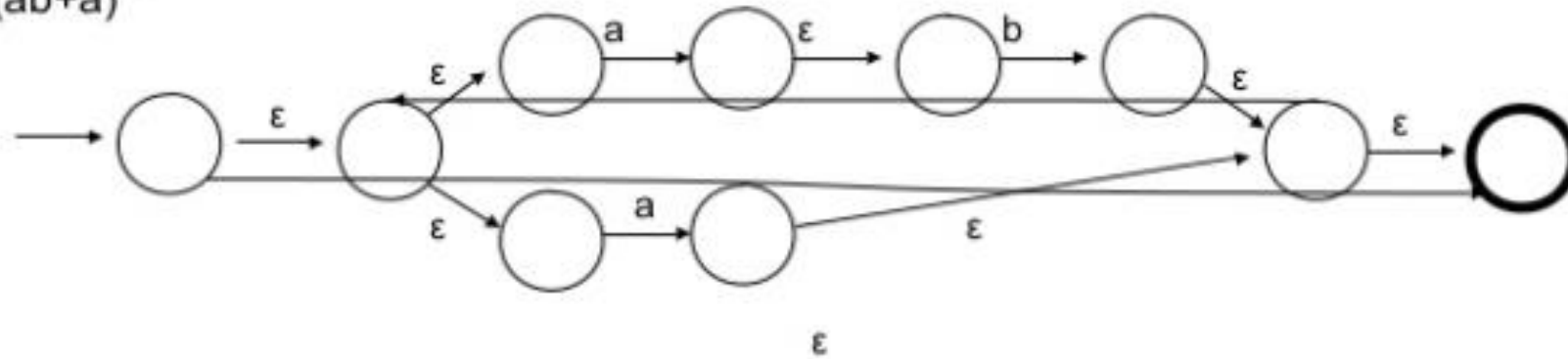
$(ab + a)^*$



$ab+a$



$(ab+a)^*$



Assignment-2

- Convert the following regular expression first into e-NFA.

1. $0+(1+0)^*00$

2. aa^*+bb^*

3. $(a+b)^*abb$

Conversion :NFA to DFA

Put ϵ -closure (s_0) as an unmarked state in to $Dstates$

While there is an unmarked state T in $Dstates$ **do**

 mark T

for each input symbol $a \in \Sigma$ **do**

$U = \epsilon\text{-closure}(\text{move}(T, a))$

if U is not in $Dstates$ **then**

 Add U as an unmarked state to $Dstates$

end if

$Dtran[T, a] = U$

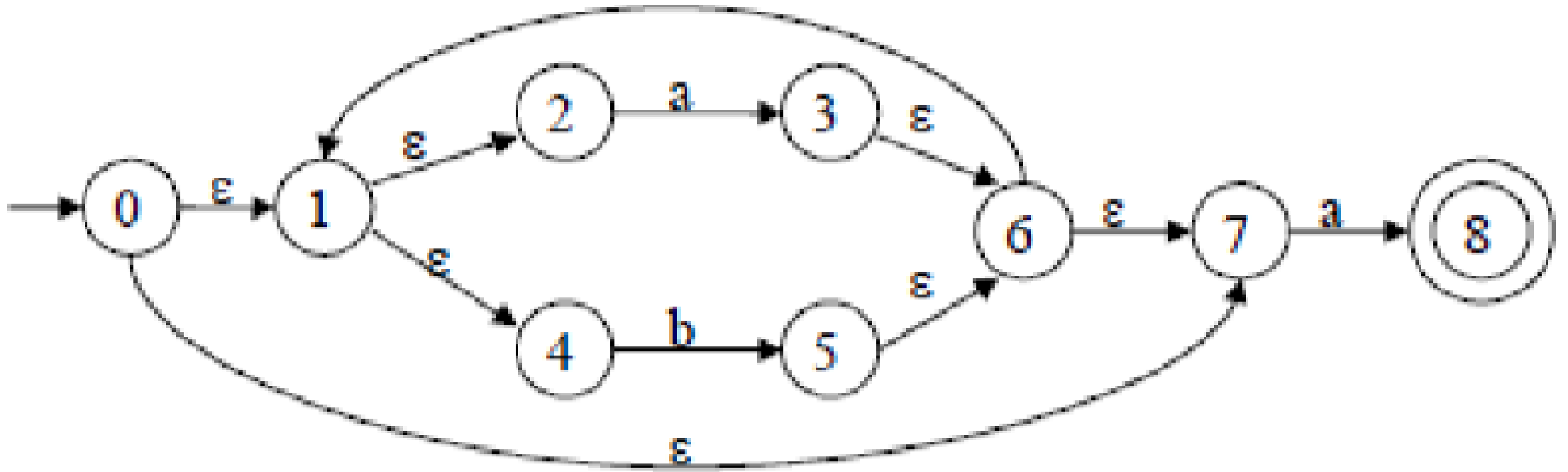
end do

end do

The algorithm produces:

$Dstates$ is the set of states of the new DFA consisting of sets of states of the NFA
 $Dtran$ is the transition table of the new DFA

Subset Construction Example (NFA to DFA)



$$S_0 = \varepsilon\text{-closure}(\{0\}) = \{0,1,2,4,7\}$$

S_0 into $Dstates$ as an unmarked state

$\Downarrow$ mark S_0

$$\varepsilon\text{-closure}(\text{move}(S_0, a)) = \varepsilon\text{-closure}(\{3,8\}) = \{1,2,3,4,6,7,8\} = S_1 \quad S_1 \text{ into } Dstates$$

$$\varepsilon\text{-closure}(\text{move}(S_0, b)) = \varepsilon\text{-closure}(\{5\}) = \{1,2,4,5,6,7\} = S_2 \quad S_2 \text{ into } Dstates$$

$$Dtran[S_0, a] \leftarrow S_1 \quad Dtran[S_0, b] \leftarrow S_2$$

$\Downarrow$ mark S_1

$$\varepsilon\text{-closure}(\text{move}(S_1, a)) = \varepsilon\text{-closure}(\{3,8\}) = \{1,2,3,4,6,7,8\} = S_1$$

$$\varepsilon\text{-closure}(\text{move}(S_1, b)) = \varepsilon\text{-closure}(\{5\}) = \{1,2,4,5,6,7\} = S_2$$

$$Dtran[S_1, a] \leftarrow S_1 \quad Dtran[S_1, b] \leftarrow S_2$$

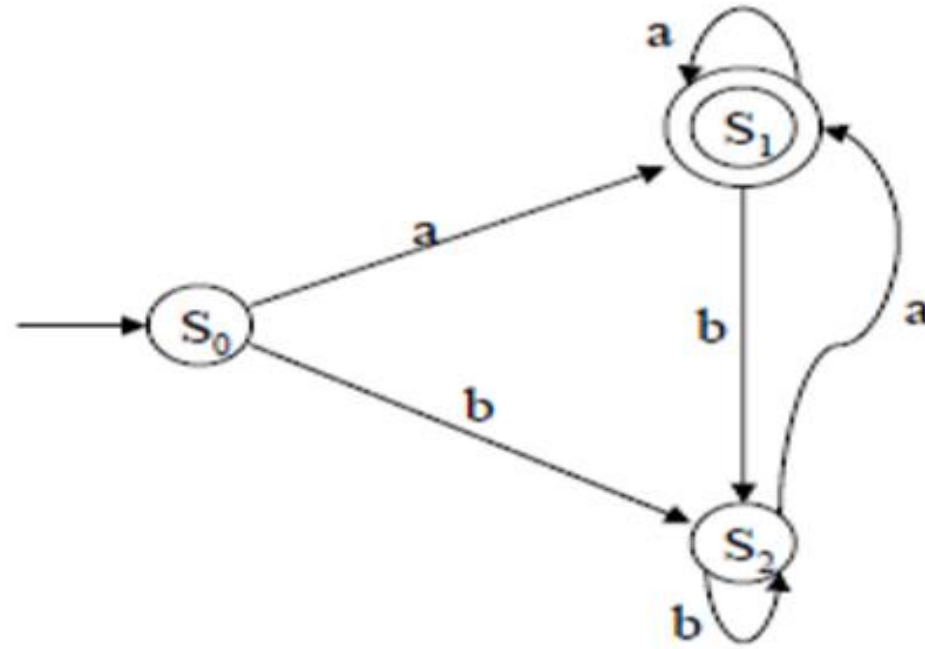
$\Downarrow$ mark S_2

$$\varepsilon\text{-closure}(\text{move}(S_2, a)) = \varepsilon\text{-closure}(\{3,8\}) = \{1,2,3,4,6,7,8\} = S_1$$

$$\varepsilon\text{-closure}(\text{move}(S_2, b)) = \varepsilon\text{-closure}(\{5\}) = \{1,2,4,5,6,7\} = S_2$$

$$Dtran[S_2, a] \leftarrow S_1 \quad Dtran[S_2, b] \leftarrow S_2$$

- S_0 is the start state of DFA since 0 is a member of $S_0 = \{0, 1, 2, 4, 7\}$
- S_1 is an accepting state of DFA since 8 is a member of $S_1 = \{1, 2, 3, 4, 6, 7, 8\}$



Assignment-2

Convert the following regular expression first into NFA and then into DFA

1. $0+(1+0)^*00$
2. aa^*+bb^*
3. $(a+b)^*abb$

$(a+b)^*abb\#$

Conversion from RE to DFA Directly

- **Conversion steps:**

✓ 1. **Augment** the given regular expression by concatenating it with special symbol #

i.e. $r \rightarrow (r) \#$ $(a+b)^*abb\#$

✓ 2. Create the syntax tree for this augmented regular expression

- In this syntax tree, all alphabet symbols (plus # and the empty string) in the augmented regular expression will be on the leaves, and all inner nodes will be the operators in that augmented regular expression.

✓ 3. Then each alphabet symbol (plus #) will be numbered (position numbers)

✓✓ 4. Traverse the tree to construct functions nullable, firstpos, lastpos, and followpos

✓✓ 5. Finally construct the DFA from the followpos

Q. $(a+b)^* \cdot a$

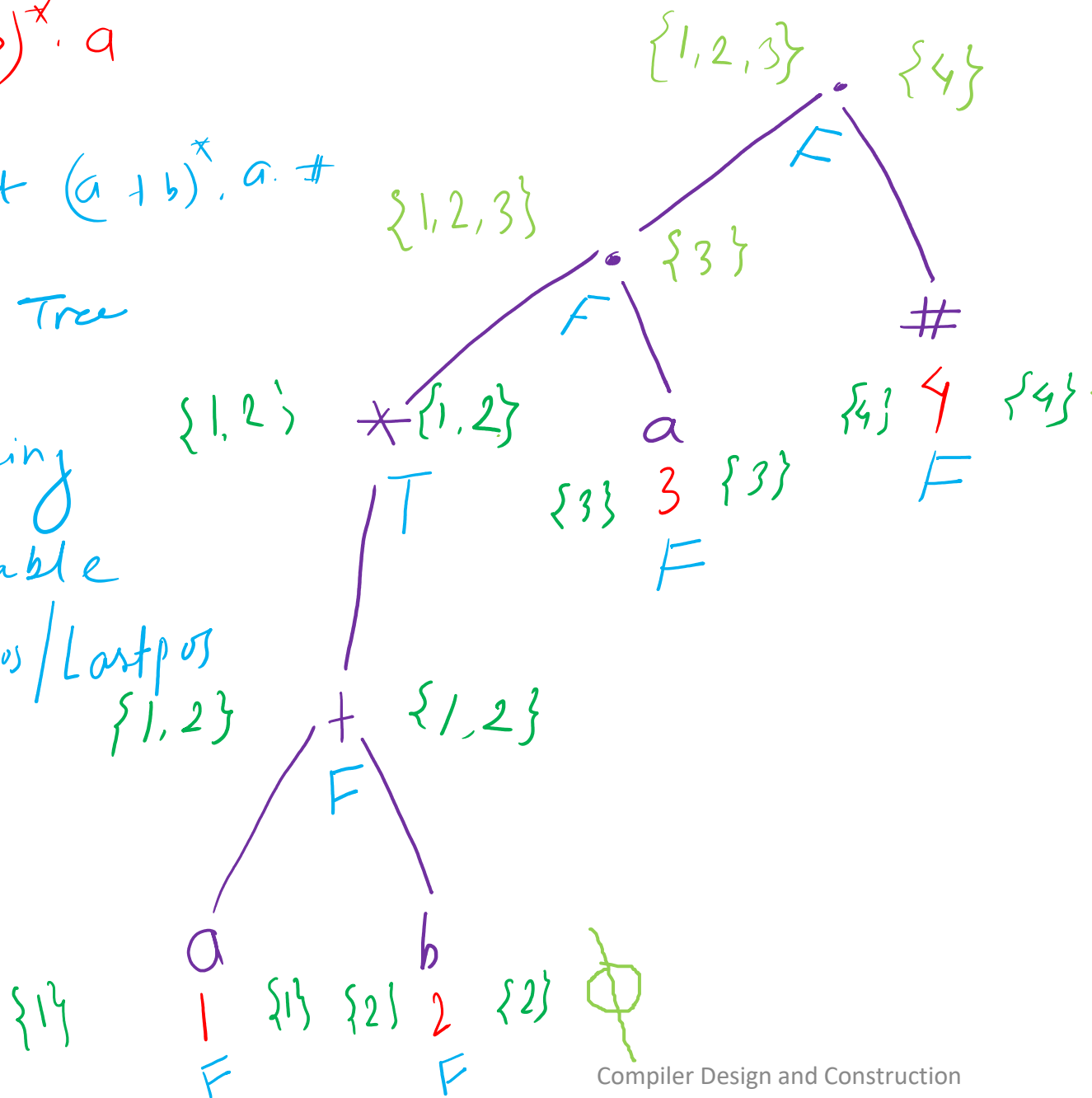
1) Augment $(a+b)^* \cdot a \cdot \#$

2) Syntax Tree

3) numbering

4) 4.1 nullable

4.2 firstpos/Lastpos



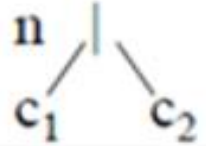
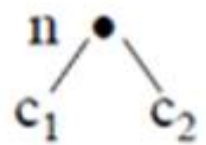
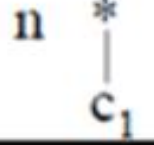
Followpos(1) = {1,2,3}

Followpos(2) = {1,2,3}

Followpos(3) = {4}

Followpos(4) = $\{\phi\}$

Rules for calculating *nullable*, *firstpos* and *lastpos*:

node <u>n</u>	<u>nullable(n)</u>	<u>firstpos(n)</u>	<u>lastpos(n)</u>
is leaf labeled ϵ	true	Φ	Φ
is leaf labeled with position i	false	$\{i\}$ (position of leaf node)	$\{i\}$
	nullable(c_1) or nullable(c_2)	$\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$	$\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$
	nullable(c_1) and nullable(c_2)	if (nullable(c_1)) then $\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$ else firstpos(c_1)	if (nullable(c_2)) then $\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$ else lastpos(c_2)
	true	$\text{firstpos}(c_1)$	$\text{lastpos}(c_1)$

node n	$\text{nullable}(n)$	$\text{firstpos}(n)$	$\text{lastpos}(n)$
is leaf labeled ε	true	Φ	Φ
is leaf labeled with position i	false	$\{i\}$ (position of leaf node)	$\{i\}$
$\begin{array}{c} n \\ \swarrow \quad \searrow \\ c_1 \quad c_2 \end{array}$	$\text{nullable}(c_1)$ or $\text{nullable}(c_2)$	$\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$	$\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$
$\begin{array}{c} n \bullet \\ \swarrow \quad \searrow \\ c_1 \quad c_2 \end{array}$	$\text{nullable}(c_1)$ and $\text{nullable}(c_2)$	if $(\text{nullable}(c_1))$ then $\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$ else $\text{firstpos}(c_1)$	if $(\text{nullable}(c_2))$ then $\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$ else $\text{lastpos}(c_2)$
$\begin{array}{c} n \\ * \\ c_1 \end{array}$	true	$\text{firstpos}(c_1)$	$\text{lastpos}(c_1)$

$(a + b)^*.a$

Algorithm to evaluate *followpos*

```
for each node  $n$  in the tree do
  if  $n$  is a cat-node with left child  $c1$  and right child  $c2$  then
    for each  $i$  in  $lastpos(c1)$  do
       $followpos(i) := followpos(i) \cup firstpos(c2)$ 
    end do
  else if  $n$  is a star-node
    for each  $i$  in  $lastpos(n)$  do
       $followpos(i) := followpos(i) \cup firstpos(n)$ 
    end do
  end if
end do
```



```

for each node  $n$  in the tree do
  if  $n$  is a cat-node with left child  $c1$  and right child  $c2$  then
    for each  $i$  in  $lastpos(c1)$  do
       $followpos(i) := followpos(i) \cup firstpos(c2)$ 
    end do
  else if  $n$  is a star-node
    for each  $i$  in  $lastpos(n)$  do
       $followpos(i) := followpos(i) \cup firstpos(n)$ 
    end do
  end if
end do

```

$(a + b) * .a.b$

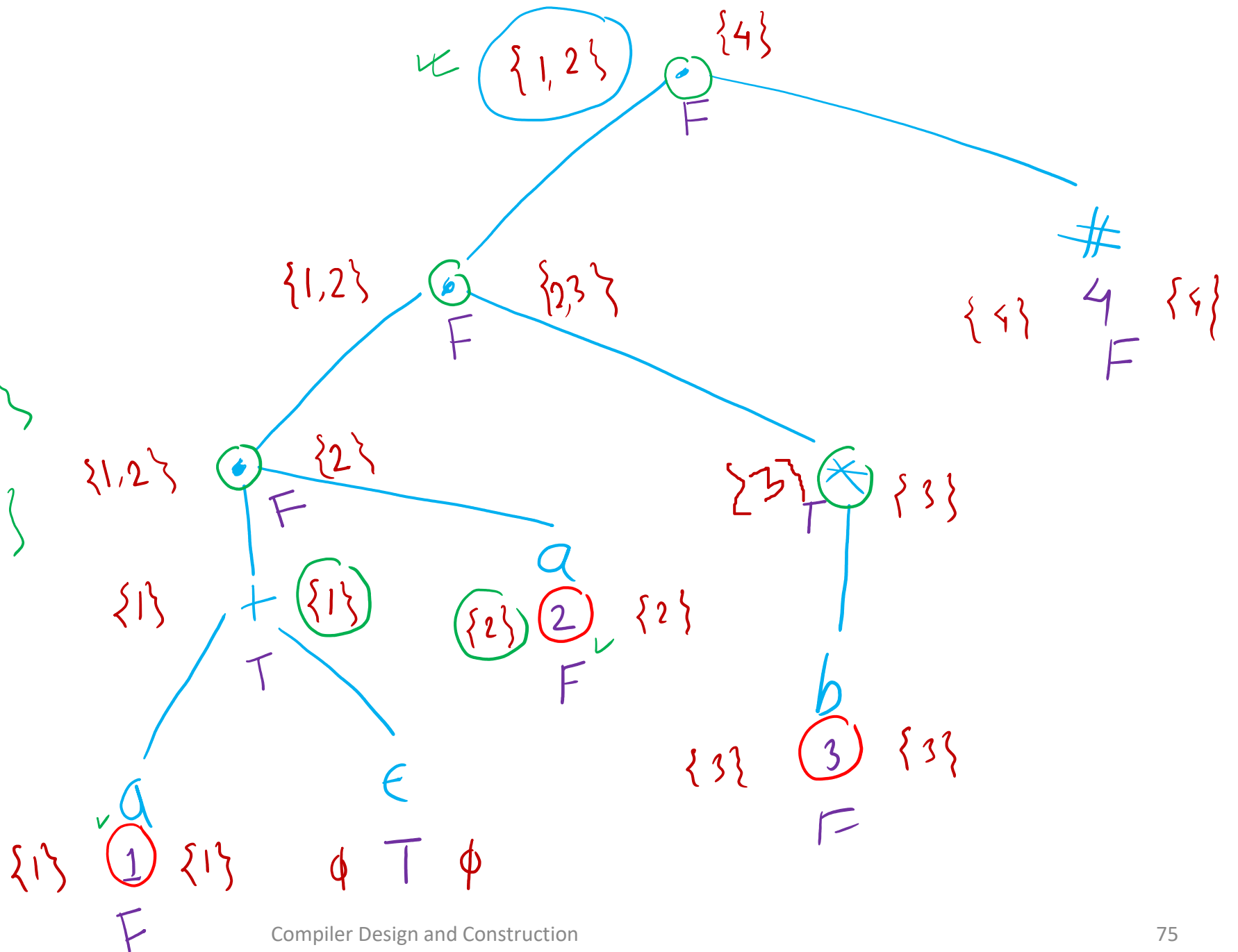
$(a + \epsilon) \cdot a \cdot b^* \cdot \#$

$\text{Followpos}(1) = \{2\}$

$\text{Followpos}(2) = \{3, 4\}$

$\text{Followpos}(3) = \{3, 4\}$

$\text{Followpos}(4) = \emptyset$



Start state = firstpos (root) = $\{\overset{\text{1}}{\text{1}}, \overset{\text{2}}{\text{2}}\} = S1$

mark S1

$\overset{\text{1}}{\text{1}}$ for a
 $\overset{\text{2}}{\text{2}}$

3 for b

mark S2

$\overset{\text{1}}{\text{1}}$ for a
3 for b

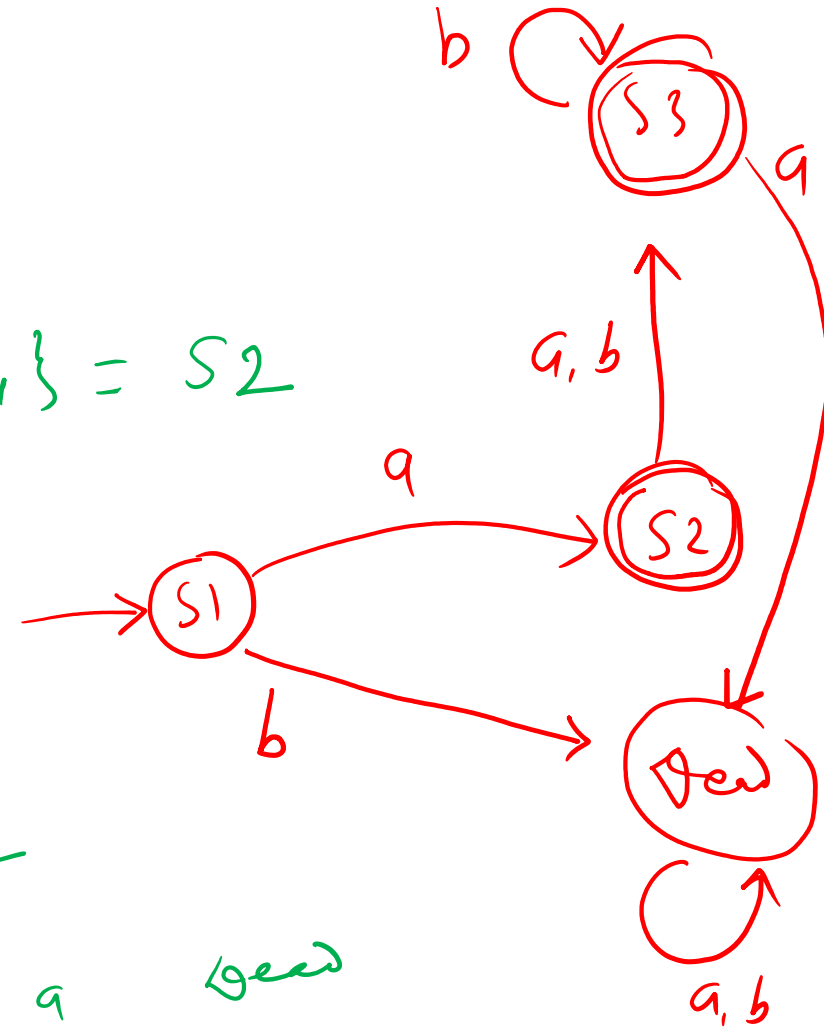
followpos (2) = {3, 4} = S3

followpos (3) = {3, 4} = S3

$(a + \epsilon) \cdot a \cdot b^*$

followpos (1) $\cup$ followpos (2) = $\{\underline{2}, 3, 4\} = S2$

dead state -



mark S3

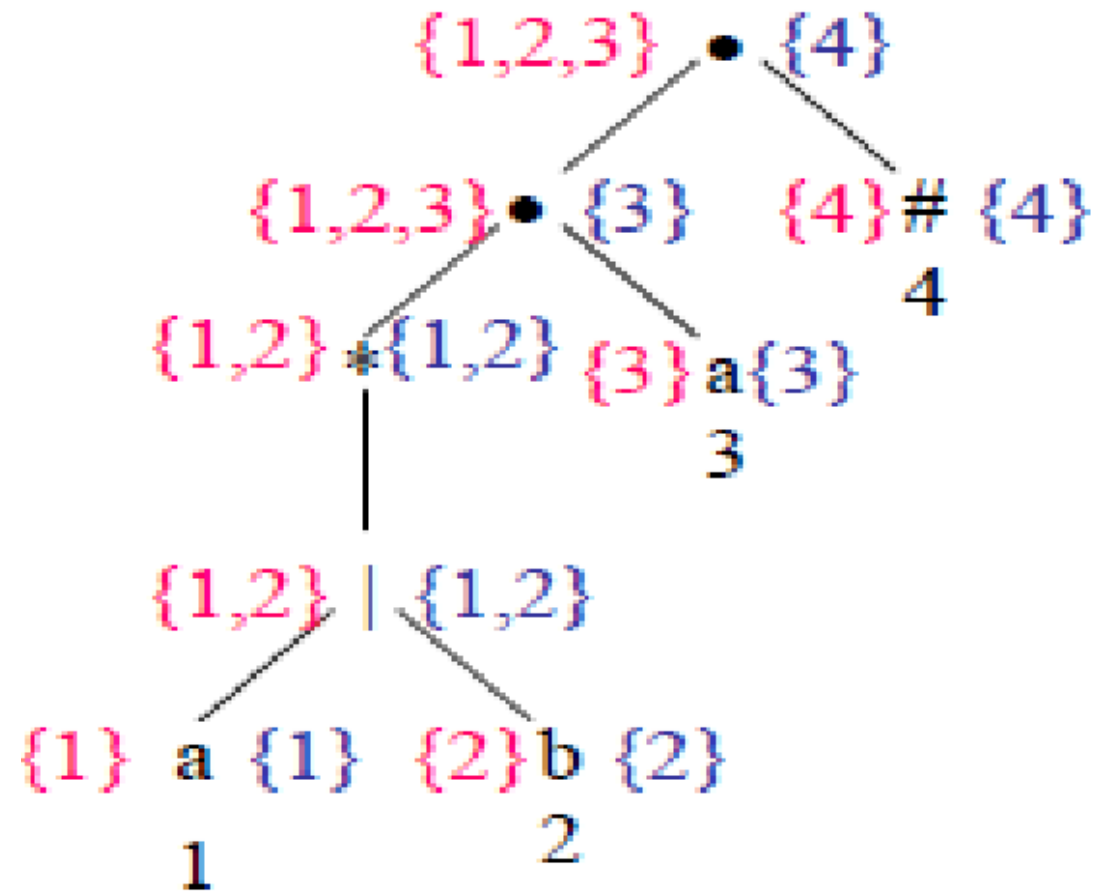
$\overset{\text{1}}{\text{1}}$ for a

3 for b

dead

followpos (3) = {3, 4} = S3

Example-1: $(a + b)^*.a$



$\text{followpos}(1) = \{1,2,3\}$

$\text{followpos}(2) = \{1,2,3\}$

$\text{followpos}(3) = \{4\}$

$\text{followpos}(4) = \{\}$

$S_1 = \text{firstpos}(\text{root}) = \{1, 2, 3\}$

mark S_1

for a : $\text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_1, a) = S_2$

for b : $\text{followpos}(2) = \{1, 2, 3\} = S_1$

$\text{move}(S_1, b) = S_1$

mark S_2

for a : $\text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_2$

$\text{move}(S_2, a) = S_2$

for b : $\text{followpos}(2) = \{1, 2, 3\} = S_1$

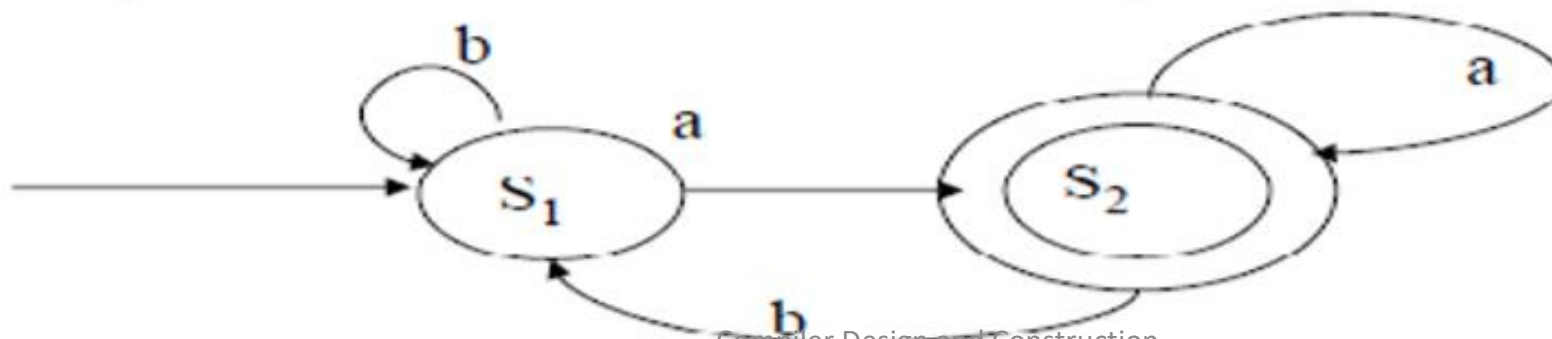
$\text{move}(S_2, b) = S_1$

Now

start state: S_1

accepting states: $\{S_2\}$

Note:- Accepting states = states containing position of # ie 4.

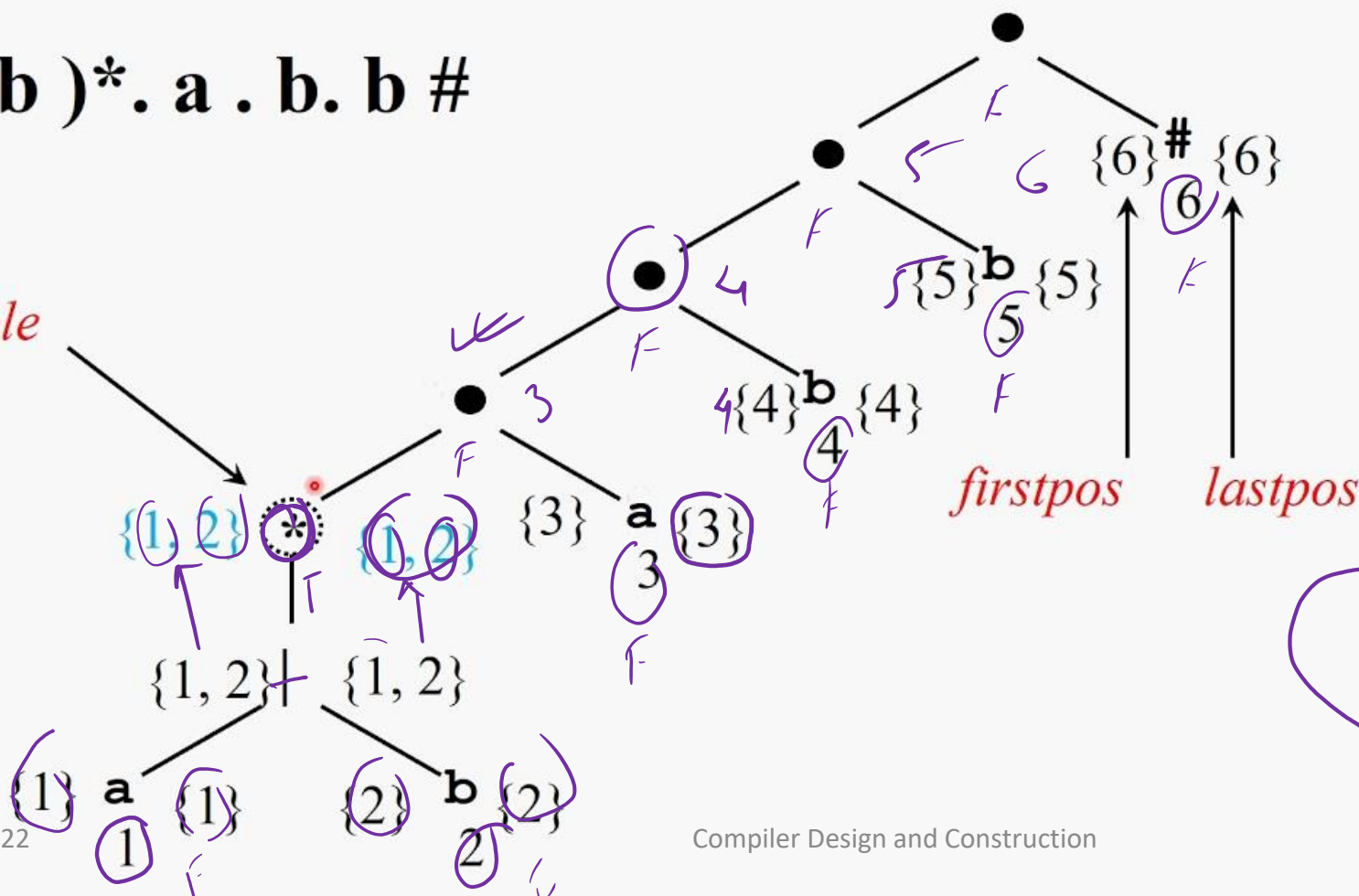


Example-2: $(a+b)^*.a.b.b$

← assignment

$(a \mid b)^*.a.b.b \#$

nullable



Followpos(1)={1,2,3}

Followpos(2)={1,2,3}

Followpos(3)={4}

Followpos(4)={5}

Followpos(5)={6}

Followpos(6)={}

For RE---- (a | ϵ) b c* #

followpos(1)= $\{2\}$ followpos(2)= $\{3,4\}$ followpos(3)= $\{3,4\}$ followpos(4)= $\{\}$

$S_1 = \text{firstpos}(\text{root}) = \{1,2\}$

mark S_1

for a: followpos(1)= $\{2\}=S_2$ $\text{move}(S_1, a) = S_2$

for b: followpos(2)= $\{3,4\}=S_3$ $\text{move}(S_1, b) = S_3$

mark S_2

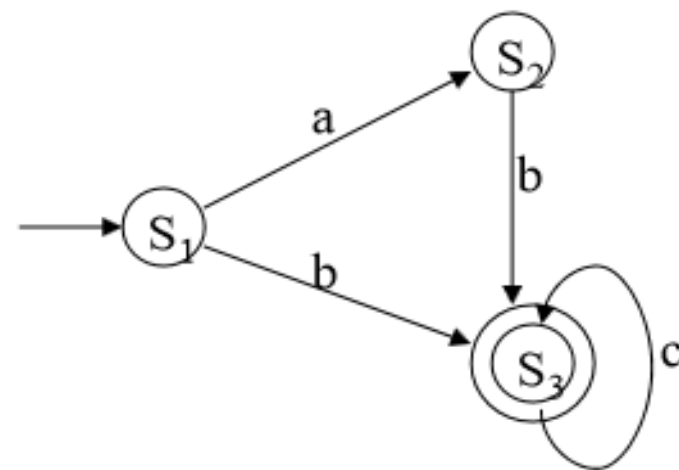
for b: followpos(2)= $\{3,4\}=S_3$ $\text{move}(S_2, b) = S_3$

mark S_3

for c: followpos(3)= $\{3,4\}=S_3$ $\text{move}(S_3, c) = S_3$

start state: S_1

accepting states: $\{S_3\}$



State Minimization in DFA

Partition the set of states into two groups: –

- G 1: set of accepting states
- G 2: set of non-accepting state

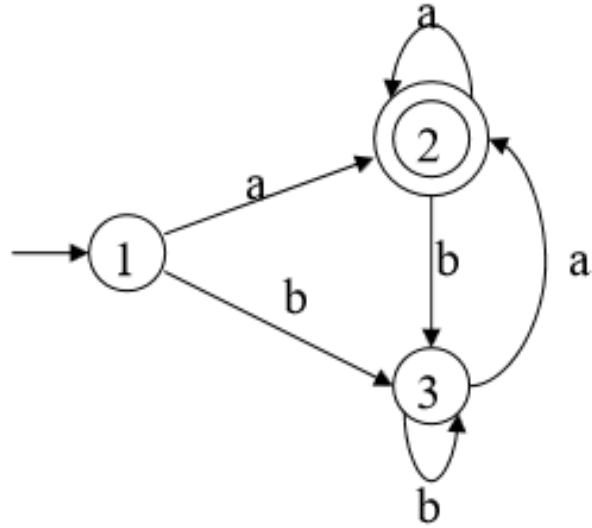
For each new group G

- partition G into subgroups such that states s_1 and s_2 are in the same group iff for all input symbols a , states s_1 and s_2 have transitions to states in the same group.

Start state of the minimized DFA is the group containing the start state of the original DFA.

Accepting states of the minimized DFA are the groups containing the accepting states of the original DFA.

Example-1



$$G_1 = \{2\}$$

$$G_2 = \{1,3\}$$

G_2 cannot be partitioned because

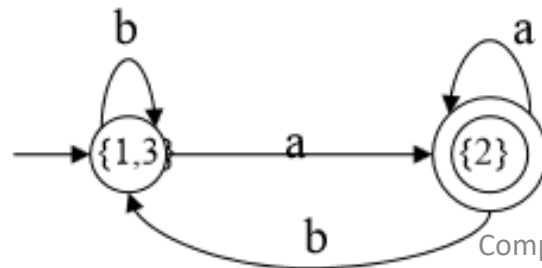
$$\text{move}(1,a)=2$$

$$\text{move}(1,b)=3$$

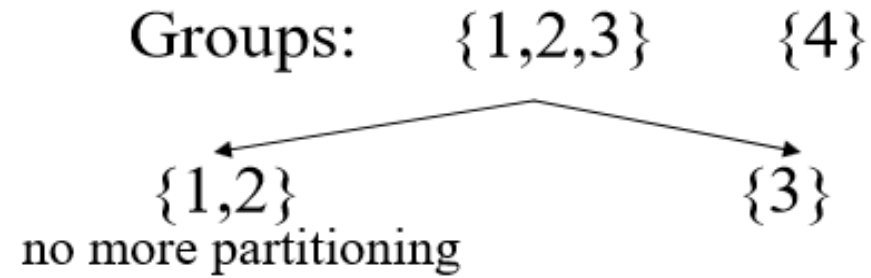
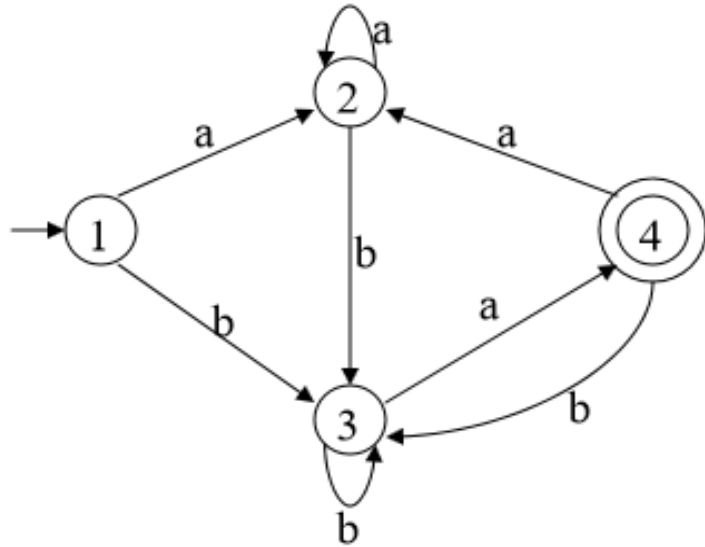
$$\text{move}(3,a)=2$$

$$\text{move}(2,b)=3$$

So, the minimized DFA (with minimum states)

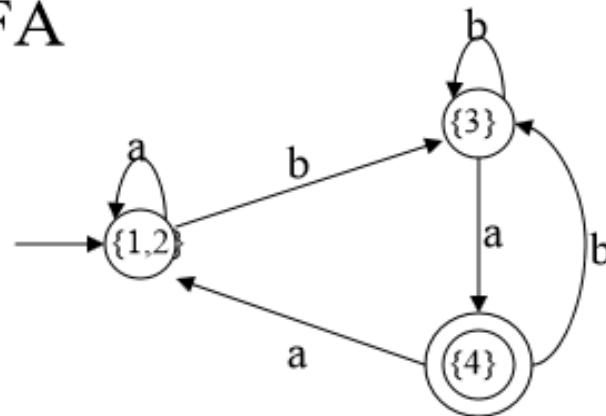


Example-2

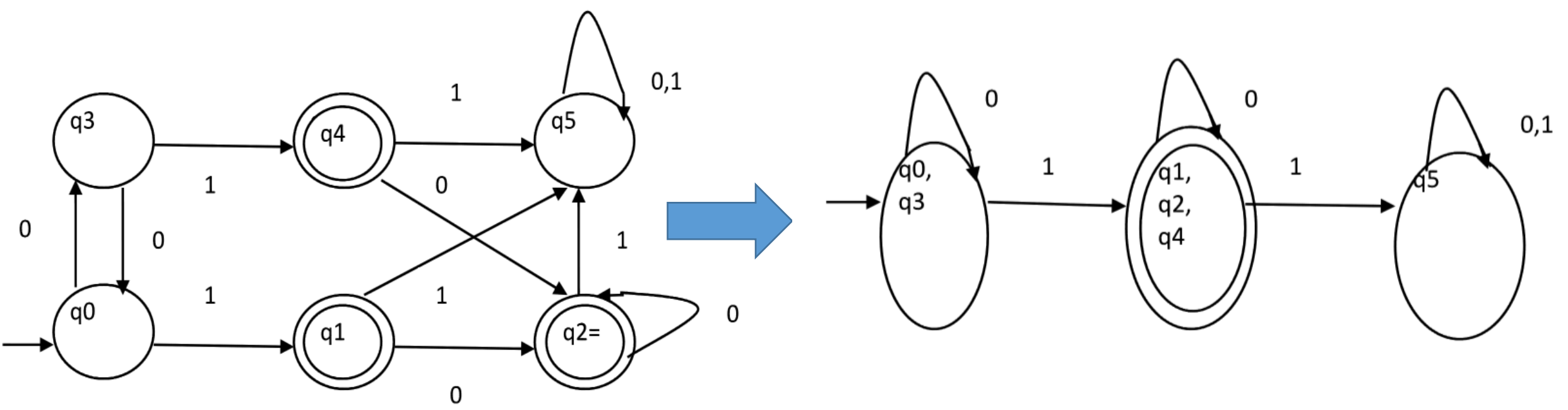


<u>a</u>	<u>b</u>
1->2	1->3
2->2	2->3
3->4	3->3

So, the minimized DFA

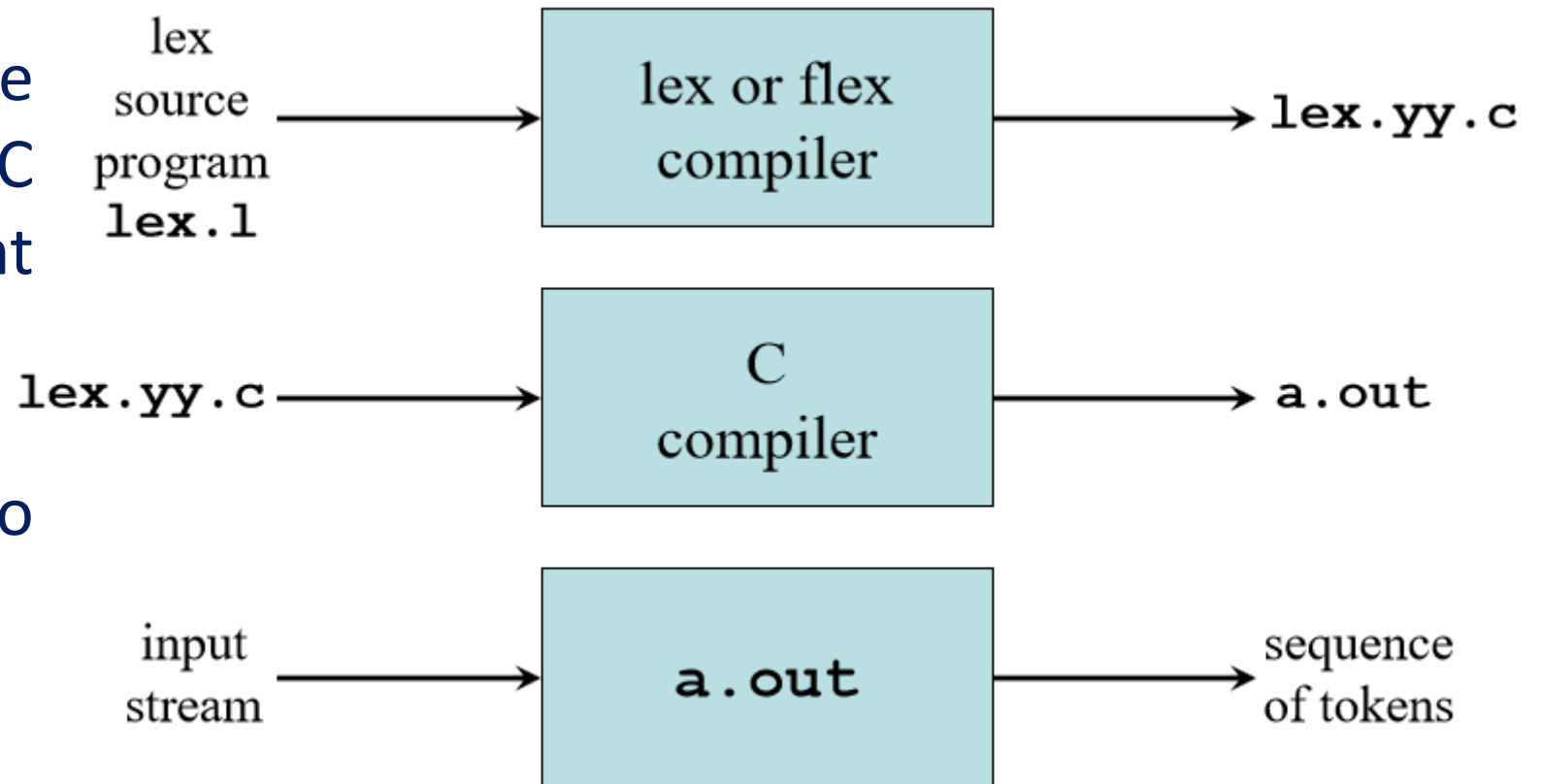


Practice Session



Flex: Language for Lexical Analyzer

- Systematically translate regular definitions into C source code for efficient scanning
- Generated code is easy to integrate in C application



Flex: Introduction

- **Flex** is a latter version of **lex**.
- It takes as its input a text file containing regular expressions, together with the action to be taken when each expression is matched.
- It produces an output file that contains C source code defining a function **yylex** that is a table-driven implementation of a DFA corresponding to the regular expressions of the input file.
- The Flex output file is then compiled with a C compiler to get an executable

Assignment-3 (Deadline: August-18,Sunday)

1. What are pattern, lexeme and tokens? Explain with examples.
2. What are Regular definitions? Explain with examples.
3. Convert the regular expression '0+ (1+0)*00' first into NFA and then into DFA using Thomson's and Subset Construction methods.
4. Minimize the DFA resulted in qn(3) using any method.
5. Convert the Regular expressions into DFA using direct method.
 - $ba(a+b)^*$
 - $(a+b)b^*c$
6. Write Short notes:
 - **Input buffering**
 - **Lex/Flex**